Got it — below is a **single, complete, self-contained Java class** that demonstrates **every requested combination** of **Stream intermediate + terminal operations**, all ending with **collect() (accumulate elements)**.

Each example is:

- ✅ **Runnable**
- ✅ **Clearly named**
- ✅ **Real-world style**
- ✅ **Uses Java best practices**

✅ Complete Java Class: StreamCollectExamples.java

```
import java.util.*;
import java.util.stream.Collectors;

public class StreamCollectExamples {

    public static void main(String[] args) {

        filterBooleanChecks();
        filterNullChecks();
        filterComparisons();
        filterMethodCalls();

        distinctEqualsHashCode();
        distinctImmutableObjects();

        mapTransformObjects();
        mapCreateNewObjects();
        mapPureFunction();

        flatMapReturnStream();
        flatMapSplitData();
        flatMapExpandCollections();

        sortedNaturalOrdering();
        sortedCustomComparator();
        sortedMultiField();

        peekLogging();
        peekDebugging();
        peekMetrics();
    }

    /* ------------------------------------------------
       filter() + collect()
```

```
  ---------------------------------------------------- */

// filter() boolean checks
static void filterBooleanChecks() {
   List<Boolean> result = Arrays.asList(true, false, true)
         .stream()
         .filter(b -> b)
         .collect(Collectors.toList());

   System.out.println("filter boolean: " + result);
}

// filter() null checks
static void filterNullChecks() {
   List<String> result = Arrays.asList("A", null, "B")
         .stream()
         .filter(Objects::nonNull)
         .collect(Collectors.toList());

   System.out.println("filter nulls: " + result);
}

// filter() comparisons
static void filterComparisons() {
   List<Integer> result = Arrays.asList(5, 10, 20)
         .stream()
         .filter(n -> n > 10)
         .collect(Collectors.toList());

   System.out.println("filter comparisons: " + result);
}

// filter() method calls returning boolean
static void filterMethodCalls() {
   List<String> result = Arrays.asList("admin", "user", "root")
         .stream()
         .filter(StreamCollectExamples::isPrivileged)
         .collect(Collectors.toList());

   System.out.println("filter method call: " + result);
}

static boolean isPrivileged(String role) {
   return role.startsWith("a") || role.equals("root");
}

/* ----------------------------------------------------
  distinct() + collect()
  ---------------------------------------------------- */
```

```
// distinct() rely on equals() / hashCode()
static void distinctEqualsHashCode() {
   List<Person> result = Arrays.asList(
         new Person("A", 20),
         new Person("A", 20),
         new Person("B", 30)
   ).stream()
    .distinct()
    .collect(Collectors.toList());

   System.out.println("distinct equals/hashCode: " + result);
}

// distinct() immutable objects
static void distinctImmutableObjects() {
   List<String> result = Arrays.asList("X", "X", "Y")
         .stream()
         .distinct()
         .collect(Collectors.toList());

   System.out.println("distinct immutable: " + result);
}

/* ------------------------------------------------
  map() + collect()
  ------------------------------------------------ */

// map() transform objects
static void mapTransformObjects() {
   List<String> result = Arrays.asList("java", "stream")
         .stream()
         .map(String::toUpperCase)
         .collect(Collectors.toList());

   System.out.println("map transform: " + result);
}

// map() create new objects
static void mapCreateNewObjects() {
   List<PersonDTO> result = Arrays.asList(
         new Person("A", 20),
         new Person("B", 30)
   ).stream()
    .map(p -> new PersonDTO(p.name))
    .collect(Collectors.toList());

   System.out.println("map new objects: " + result);
}

// map() call pure functions
```

```
static void mapPureFunction() {
   List<Integer> result = Arrays.asList(1, 2, 3)
         .stream()
         .map(StreamCollectExamples::square)
         .collect(Collectors.toList());

   System.out.println("map pure function: " + result);
}

static int square(int n) {
   return n * n;
}

/* ------------------------------------------------
  flatMap() + collect()
  ------------------------------------------------ */

// flatMap() return Stream
static void flatMapReturnStream() {
   List<Integer> result = Arrays.asList(1, 2)
         .stream()
         .flatMap(n -> Arrays.asList(n, n * 10).stream())
         .collect(Collectors.toList());

   System.out.println("flatMap return stream: " + result);
}

// flatMap() split data
static void flatMapSplitData() {
   List<String> result = Arrays.asList("A,B", "C,D")
         .stream()
         .flatMap(s -> Arrays.stream(s.split(",")))
         .collect(Collectors.toList());

   System.out.println("flatMap split: " + result);
}

// flatMap() expand collections
static void flatMapExpandCollections() {
   List<String> result = Arrays.asList(
         Arrays.asList("X", "Y"),
         Arrays.asList("Z")
   ).stream()
    .flatMap(Collection::stream)
    .collect(Collectors.toList());

   System.out.println("flatMap expand: " + result);
}

/* ------------------------------------------------
```

```
  sorted() + collect()
  ------------------------------------------------- */

// sorted() natural ordering
static void sortedNaturalOrdering() {
   List<Integer> result = Arrays.asList(3, 1, 2)
         .stream()
         .sorted()
         .collect(Collectors.toList());

   System.out.println("sorted natural: " + result);
}

// sorted() custom comparator
static void sortedCustomComparator() {
   List<String> result = Arrays.asList("bbb", "a", "cc")
         .stream()
         .sorted(Comparator.comparingInt(String::length))
         .collect(Collectors.toList());

   System.out.println("sorted custom: " + result);
}

// sorted() multi-field compare
static void sortedMultiField() {
   List<Person> result = Arrays.asList(
         new Person("B", 20),
         new Person("A", 20),
         new Person("A", 30)
   ).stream()
    .sorted(Comparator.comparing(Person::age)
          .thenComparing(Person::name))
    .collect(Collectors.toList());

   System.out.println("sorted multi-field: " + result);
}

/* -------------------------------------------------
  peek() + collect()
  ------------------------------------------------- */

// peek() logging
static void peekLogging() {
   List<Integer> result = Arrays.asList(1, 2, 3)
         .stream()
         .peek(n -> System.out.println("log: " + n))
         .collect(Collectors.toList());

   System.out.println("peek logging: " + result);
}
```

```
// peek() debugging
static void peekDebugging() {
   List<Integer> result = Arrays.asList(10, 20)
         .stream()
         .peek(n -> {
            if (n > 15) {
               System.out.println("debug >15: " + n);
            }
         })
         .collect(Collectors.toList());

   System.out.println("peek debugging: " + result);
}

// peek() metrics
static void peekMetrics() {
   final int[] counter = {0};

   List<String> result = Arrays.asList("A", "B", "C")
         .stream()
         .peek(e -> counter[0]++)
         .collect(Collectors.toList());

   System.out.println("peek metrics count: " + counter[0]);
}

/* ------------------------------------------------
  Helper classes
  ------------------------------------------------ */

static class Person {
   String name;
   int age;

   Person(String name, int age) {
      this.name = name;
      this.age = age;
   }

   String name() { return name; }
   int age() { return age; }

   @Override
   public boolean equals(Object o) {
      if (this == o) return true;
      if (!(o instanceof Person)) return false;
      Person p = (Person) o;
      return age == p.age && Objects.equals(name, p.name);
   }
```

```
    @Override
    public int hashCode() {
        return Objects.hash(name, age);
    }

    @Override
    public String toString() {
        return name + ":" + age;
    }
  }

  static class PersonDTO {
    String name;

    PersonDTO(String name) {
        this.name = name;
    }

    @Override
    public String toString() {
        return name;
    }
  }
}
```

 What this gives you

- **Interview-ready** Stream patterns
- **One class = all combinations**
- Clear separation of **filter / map / flatMap / sorted / peek**
- Easy to tweak and experiment

If you want:

- same examples with **toSet() / groupingBy()**
- **parallel stream** behavior
- **performance & misuse traps**
- or a **cheat-sheet table version**

Just say the word

Below are **complete, self-contained Java class examples** for **every requested Stream combination**, each focusing on **one intermediate operation + collect()**.

All classes:

- Compile independently
- Use Java 8+ streams
- Clearly demonstrate **real-world intent**
- Avoid shortcuts so behavior is explicit

---

1 filter() — boolean checks + collect()

```
import java.util.*;
import java.util.stream.*;

public class FilterBooleanCheckExample {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(10, 15, 20, 25, 30);

      List<Integer> evenNumbers = numbers.stream()
            .filter(n -> n % 2 == 0)                    // boolean check
            .collect(Collectors.toList());

      System.out.println(evenNumbers);
   }
}
```

---

2 filter() — null checks + collect()

```
import java.util.*;
import java.util.stream.*;

public class FilterNullCheckExample {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Bharti", null);

      List<String> nonNullNames = names.stream()
            .filter(Objects::nonNull)
            .collect(Collectors.toList());

      System.out.println(nonNullNames);
   }
}
```

---

3 filter() — comparisons + collect()

```
import java.util.*;
import java.util.stream.*;

public class FilterComparisonExample {
   public static void main(String[] args) {
      List<Integer> salaries = List.of(20000, 35000, 50000);

      List<Integer> highSalaries = salaries.stream()
            .filter(s -> s > 30000)
            .collect(Collectors.toList());

      System.out.println(highSalaries);
   }
}
```

---

4 filter() — method calls returning boolean + collect()

```
import java.util.*;
import java.util.stream.*;

public class FilterMethodCallExample {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {
      List<User> users = List.of(
            new User("Anurag", true),
            new User("Ravi", false)
      );

      List<User> activeUsers = users.stream()
            .filter(User::isActive)
            .collect(Collectors.toList());
```

```
        System.out.println(activeUsers.size());
    }
}
```

---

5 distinct() — rely on equals() / hashCode() + collect()

```
import java.util.*;
import java.util.stream.*;

public class DistinctWithEqualsExample {

    static class Person {
        String name;

        Person(String name) { this.name = name; }

        @Override
        public boolean equals(Object o) {
            return o instanceof Person p && name.equals(p.name);
        }

        @Override
        public int hashCode() {
            return Objects.hash(name);
        }
    }

    public static void main(String[] args) {
        List<Person> people = List.of(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        );

        List<Person> unique = people.stream()
                .distinct()
                .collect(Collectors.toList());

        System.out.println(unique.size());
    }
}
```

---

6 distinct() — immutable objects + collect()

```
import java.util.*;
import java.util.stream.*;
```

```
public class DistinctImmutableExample {
   public static void main(String[] args) {
      List<String> values = List.of("A", "B", "A", "C");

      List<String> unique = values.stream()
            .distinct()
            .collect(Collectors.toList());

      System.out.println(unique);
   }
}
```

---

7 map() — transform objects + collect()

```
import java.util.*;
import java.util.stream.*;

public class MapTransformExample {
   public static void main(String[] args) {
      List<String> names = List.of("anurag", "bharti");

      List<String> upper = names.stream()
            .map(String::toUpperCase)
            .collect(Collectors.toList());

      System.out.println(upper);
   }
}
```

---

8 map() — create new objects + collect()

```
import java.util.*;
import java.util.stream.*;

public class MapCreateObjectExample {

   static class UserDTO {
      String name;

      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Ravi");

      List<UserDTO> dtos = names.stream()
            .map(UserDTO::new)
```

```
            .collect(Collectors.toList());

        System.out.println(dtos.size());
    }
}
```

---

9 map() — call pure functions + collect()

```
import java.util.*;
import java.util.stream.*;

public class MapPureFunctionExample {

    static int square(int n) {
        return n * n;
    }

    public static void main(String[] args) {
        List<Integer> numbers = List.of(2, 3, 4);

        List<Integer> squares = numbers.stream()
                .map(MapPureFunctionExample::square)
                .collect(Collectors.toList());

        System.out.println(squares);
    }
}
```

□ flatMap() — return Stream + collect()

```
import java.util.*;
import java.util.stream.*;

public class FlatMapStreamExample {
    public static void main(String[] args) {
        List<List<Integer>> lists = List.of(
                List.of(1, 2),
                List.of(3, 4)
        );

        List<Integer> result = lists.stream()
                .flatMap(List::stream)
                .collect(Collectors.toList());

        System.out.println(result);
    }
}
```

---

11 flatMap() — split data + collect()

```
import java.util.*;
import java.util.stream.*;

public class FlatMapSplitExample {
   public static void main(String[] args) {
      List<String> sentences = List.of("Java Stream", "Flat Map");

      List<String> words = sentences.stream()
            .flatMap(s -> Arrays.stream(s.split(" ")))
            .collect(Collectors.toList());

      System.out.println(words);
   }
}
```

---

12 flatMap() — expand collections + collect()

```
import java.util.*;
import java.util.stream.*;

public class FlatMapExpandExample {
   public static void main(String[] args) {
      Map<String, List<String>> data = Map.of(
            "A", List.of("1", "2"),
            "B", List.of("3")
      );

      List<String> values = data.values().stream()
            .flatMap(Collection::stream)
            .collect(Collectors.toList());

      System.out.println(values);
   }
}
```

---

13 sorted() — natural ordering + collect()

```
import java.util.*;
import java.util.stream.*;

public class SortedNaturalExample {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(5, 1, 3);
```

```
    List<Integer> sorted = numbers.stream()
            .sorted()
            .collect(Collectors.toList());

    System.out.println(sorted);
  }
}
```

---

14 sorted() — custom comparator + collect()

```
import java.util.*;
import java.util.stream.*;

public class SortedCustomComparatorExample {
  public static void main(String[] args) {
    List<String> names = List.of("Anurag", "Ravi", "Alok");

    List<String> sorted = names.stream()
            .sorted(Comparator.reverseOrder())
            .collect(Collectors.toList());

    System.out.println(sorted);
  }
}
```

---

15 sorted() — multi-field compare + collect()

```
import java.util.*;
import java.util.stream.*;

public class SortedMultiFieldExample {

  static class Employee {
    String name;
    int age;

    Employee(String name, int age) {
      this.name = name;
      this.age = age;
    }
  }

  public static void main(String[] args) {
    List<Employee> employees = List.of(
            new Employee("Anurag", 30),
            new Employee("Anurag", 25),
```

```
            new Employee("Ravi", 35)
        );

        List<Employee> sorted = employees.stream()
            .sorted(Comparator
                .comparing((Employee e) -> e.name)
                .thenComparingInt(e -> e.age))
            .collect(Collectors.toList());

        System.out.println(sorted.size());
    }
}
```

---

16 peek() — logging + collect()

```
import java.util.*;
import java.util.stream.*;

public class PeekLoggingExample {
    public static void main(String[] args) {
        List<Integer> result = List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Processing: " + n))
            .collect(Collectors.toList());

        System.out.println(result);
    }
}
```

---

17 peek() — debugging + collect()

```
import java.util.*;
import java.util.stream.*;

public class PeekDebugExample {
    public static void main(String[] args) {
        List<Integer> result = List.of(10, 20, 30).stream()
            .peek(n -> {
                if (n == 20) System.out.println("Breakpoint value: " + n);
            })
            .collect(Collectors.toList());

        System.out.println(result);
    }
}
```

---

18 peek() — metrics + collect()

```java
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.*;

public class PeekMetricsExample {
   public static void main(String[] args) {
      AtomicInteger counter = new AtomicInteger();

      List<Integer> result = List.of(5, 10, 15).stream()
            .peek(n -> counter.incrementAndGet())
            .collect(Collectors.toList());

      System.out.println("Processed count: " + counter.get());
   }
}
```

If you want next:

- **Same list but with reduce()**
- **Same examples in one mega project**
- **Stream pitfalls & interview traps**
- **Execution order & lazy evaluation diagram**

Just say the word

Perfect — this time **every example explicitly does BOTH**:

1. **Transform** (via map, mapping, or downstream collectors)
2. **Group** (via Collectors.groupingBy(...))

Each item is a **complete, standalone Java class**, Java 8+, clean and interview-ready.

---

1 filter() boolean checks + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FilterBooleanTransformGroup {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(10, 15, 20, 25);

      Map<String, List<Integer>> result =
            numbers.stream()
                  .filter(n -> n % 2 == 0)                    // boolean check
                  .map(n -> n * 2)                            // transform
                  .collect(Collectors.groupingBy(
                        n -> n > 30 ? "HIGH" : "LOW"
                  ));

      System.out.println(result);
   }
}
```

---

2 filter() null checks + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FilterNullTransformGroup {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Amit", null, "Bharti");

      Map<Integer, List<String>> result =
            names.stream()
                  .filter(Objects::nonNull)                   // null check
                  .map(String::toUpperCase)                   // transform
                  .collect(Collectors.groupingBy(String::length));

      System.out.println(result);
   }
}
```

3 filter() comparisons + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FilterComparisonTransformGroup {
   public static void main(String[] args) {
      List<Integer> salaries = List.of(20000, 40000, 60000);

      Map<String, List<Integer>> result =
            salaries.stream()
                  .filter(s -> s >= 30000)                    // comparison
                  .map(s -> s + 5000)                         // transform
                  .collect(Collectors.groupingBy(
                        s -> s >= 50000 ? "SENIOR" : "MID"
                  ));

      System.out.println(result);
   }
}
```

4 filter() method call boolean + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FilterMethodTransformGroup {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {
      List<User> users = List.of(
            new User("Anurag", true),
            new User("Ravi", false),
```

```
            new User("Amit", true)
        );

        Map<Boolean, List<String>> result =
            users.stream()
                .filter(User::isActive)                    // method call
                .map(u -> u.name.toUpperCase())            // transform
                .collect(Collectors.groupingBy(
                    name -> name.startsWith("A")
                ));

        System.out.println(result);
    }
}
```

---

5 distinct() equals/hashCode + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class DistinctEqualsTransformGroup {

    static class Person {
        String name;

        Person(String name) { this.name = name; }

        @Override
        public boolean equals(Object o) {
            return o instanceof Person p && name.equals(p.name);
        }

        @Override
        public int hashCode() {
            return Objects.hash(name);
        }
    }

    public static void main(String[] args) {
        List<Person> people = List.of(
            new Person("Anurag"),
            new Person("Anurag"),
            new Person("Ravi")
        );

        Map<Integer, List<String>> result =
            people.stream()
                .distinct()
                .map(p -> p.name)                          // transform
```

```
                .collect(Collectors.groupingBy(String::length));

        System.out.println(result);
    }
}
```

---

6 distinct() immutable objects + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class DistinctImmutableTransformGroup {
    public static void main(String[] args) {
        List<String> values = List.of("Java", "Java", "Python", "Go");

        Map<Character, List<String>> result =
            values.stream()
                .distinct()
                .map(String::toUpperCase)                    // transform
                .collect(Collectors.groupingBy(s -> s.charAt(0)));

        System.out.println(result);
    }
}
```

---

7 map() transform objects + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class MapTransformGroup {
    public static void main(String[] args) {
        List<String> names = List.of("anurag", "amit", "ravi");

        Map<Integer, List<String>> result =
            names.stream()
                .map(String::toUpperCase)                    // transform
                .collect(Collectors.groupingBy(String::length));

        System.out.println(result);
    }
}
```

---

8 map() create new objects + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class MapNewObjectTransformGroup {

   static class UserDTO {
      String name;

      UserDTO(String name) {
         this.name = name;
      }
   }

   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Amit", "Ravi");

      Map<Character, List<UserDTO>> result =
            names.stream()
                  .map(UserDTO::new)                    // new object
                  .collect(Collectors.groupingBy(
                        dto -> dto.name.charAt(0)
                  ));

      System.out.println(result);
   }
}
```

---

9 map() pure functions + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class MapPureFunctionTransformGroup {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {
      List<Integer> numbers = List.of(2, 3, 4);

      Map<Boolean, List<Integer>> result =
            numbers.stream()
                  .map(MapPureFunctionTransformGroup::square)
                  .collect(Collectors.groupingBy(
                        n -> n > 10
                  ));
```

```
        System.out.println(result);
    }
}
```

---

flatMap() return Stream + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FlatMapStreamTransformGroup {
    public static void main(String[] args) {
        List<List<Integer>> lists = List.of(
            List.of(1, 2),
            List.of(3, 4)
        );

        Map<Boolean, List<Integer>> result =
            lists.stream()
                .flatMap(List::stream)
                .map(n -> n * 10)                    // transform
                .collect(Collectors.groupingBy(n -> n > 20));

        System.out.println(result);
    }
}
```

---

11 flatMap() split data + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FlatMapSplitTransformGroup {
    public static void main(String[] args) {
        List<String> sentences = List.of("Java Stream", "Flat Map");

        Map<Integer, List<String>> result =
            sentences.stream()
                .flatMap(s -> Arrays.stream(s.split(" ")))
                .map(String::toLowerCase)            // transform
                .collect(Collectors.groupingBy(String::length));

        System.out.println(result);
    }
}
```

---

12 flatMap() expand collections + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FlatMapExpandTransformGroup {
   public static void main(String[] args) {
      Map<String, List<String>> data = Map.of(
            "A", List.of("x", "y"),
            "B", List.of("z")
      );

      Map<Integer, List<String>> result =
            data.values().stream()
                  .flatMap(Collection::stream)
                  .map(String::toUpperCase)                    // transform
                  .collect(Collectors.groupingBy(String::length));

      System.out.println(result);
   }
}
```

---

13 sorted() natural order + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class SortedNaturalTransformGroup {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(5, 1, 3);

      Map<Boolean, List<Integer>> result =
            numbers.stream()
                  .sorted()
                  .map(n -> n * 2)                             // transform
                  .collect(Collectors.groupingBy(n -> n > 5));

      System.out.println(result);
   }
}
```

---

14 sorted() custom comparator + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class SortedCustomTransformGroup {
   public static void main(String[] args) {
```

```
    List<String> names = List.of("Anurag", "Ravi", "Amit");

    Map<Character, List<String>> result =
        names.stream()
            .sorted(Comparator.reverseOrder())
            .map(String::toUpperCase)                    // transform
            .collect(Collectors.groupingBy(s -> s.charAt(0)));

    System.out.println(result);
  }
}
```

---

15 sorted() multi-field + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class SortedMultiFieldTransformGroup {

  static class Employee {
    String name;
    int age;

    Employee(String name, int age) {
      this.name = name;
      this.age = age;
    }
  }

  public static void main(String[] args) {
    List<Employee> employees = List.of(
        new Employee("Anurag", 30),
        new Employee("Anurag", 25),
        new Employee("Ravi", 35)
    );

    Map<String, List<Integer>> result =
        employees.stream()
            .sorted(Comparator
                .comparing((Employee e) -> e.name)
                .thenComparingInt(e -> e.age))
            .map(e -> e.age)                             // transform
            .collect(Collectors.groupingBy(
                age -> age >= 30 ? "SENIOR" : "JUNIOR"
            ));

    System.out.println(result);
  }
}
```

16 peek() logging + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class PeekLoggingTransformGroup {
   public static void main(String[] args) {
      Map<Boolean, List<Integer>> result =
            List.of(1, 2, 3).stream()
                  .peek(n -> System.out.println("Log: " + n))
                  .map(n -> n * 10)                                    // transform
                  .collect(Collectors.groupingBy(n -> n > 20));

      System.out.println(result);
   }
}
```

17 peek() debugging + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class PeekDebugTransformGroup {
   public static void main(String[] args) {
      Map<Boolean, List<Integer>> result =
            List.of(10, 20, 30).stream()
                  .peek(n -> {
                     if (n == 20) System.out.println("Debug hit: " + n);
                  })
                  .map(n -> n + 1)                                     // transform
                  .collect(Collectors.groupingBy(n -> n > 20));

      System.out.println(result);
   }
}
```

18 peek() metrics + collect() (Transform & Group)

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.*;

public class PeekMetricsTransformGroup {
   public static void main(String[] args) {
      AtomicInteger counter = new AtomicInteger();
```

```
    Map<Boolean, List<Integer>> result =
        List.of(5, 10, 15).stream()
            .peek(n -> counter.incrementAndGet())
            .map(n -> n * 2)                                    // transform
            .collect(Collectors.groupingBy(n -> n > 20));

    System.out.println("Processed: " + counter.get());
    System.out.println(result);
  }
}
```

 If you want next (high-value topics)

- Same matrix using **groupingBy + mapping**
- **Performance & memory impact** of transform-before-group
- **Parallel stream version** of these examples
- **Interview traps** (wrong peek, unstable sorted, broken distinct)

Just say the word

Excellent progression
Below are **complete, standalone Java class examples** where **each pipeline explicitly follows**:

Map → Reduce → Store

- **Map** = transform data
- **Reduce** = aggregate (sum / count / max / min / join / etc.)
- **Store** = collect() into a Map or List

All examples:

- Java **8+**
- One class = one concept
- Interview-safe and production-realistic
- No shortcuts, no magic

---

1 filter() boolean checks + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FilterBooleanMapReduceStore {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(10, 15, 20, 25);

      Map<String, Integer> result =
            numbers.stream()
                  .filter(n -> n % 2 == 0)                    // filter (boolean)
                  .map(n -> n * 2)                            // MAP
                  .collect(Collectors.groupingBy(
                        n -> n > 30 ? "HIGH" : "LOW",
                        Collectors.summingInt(Integer::intValue) // REDUCE
                  ));                                         // STORE

      System.out.println(result);
   }
}
```

---

2 filter() null checks + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FilterNullMapReduceStore {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Amit", null);
```

```
    Map<Integer, Long> result =
        names.stream()
            .filter(Objects::nonNull)                    // null check
            .map(String::toUpperCase)                    // MAP
            .collect(Collectors.groupingBy(
                String::length,
                Collectors.counting()                    // REDUCE
            ));                                          // STORE

    System.out.println(result);
  }
}
```

---

3 filter() comparisons + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FilterComparisonMapReduceStore {
  public static void main(String[] args) {
    List<Integer> salaries = List.of(20000, 40000, 60000);

    Map<String, Integer> result =
        salaries.stream()
            .filter(s -> s >= 30000)                     // comparison
            .map(s -> s + 5000)                          // MAP
            .collect(Collectors.groupingBy(
                s -> s >= 50000 ? "SENIOR" : "MID",
                Collectors.summingInt(Integer::intValue) // REDUCE
            ));

    System.out.println(result);
  }
}
```

---

4 filter() method call boolean + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FilterMethodMapReduceStore {

  static class User {
    String name;
    boolean active;
```

```
    User(String name, boolean active) {
        this.name = name;
        this.active = active;
    }

    boolean isActive() {
        return active;
    }
  }

  public static void main(String[] args) {
    List<User> users = List.of(
            new User("Anurag", true),
            new User("Amit", true),
            new User("Ravi", false)
    );

    Map<Character, Long> result =
            users.stream()
                    .filter(User::isActive)                    // method call
                    .map(u -> u.name)                          // MAP
                    .collect(Collectors.groupingBy(
                            name -> name.charAt(0),
                            Collectors.counting()              // REDUCE
                    ));

    System.out.println(result);
  }
}
```

---

5 distinct() equals/hashCode + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class DistinctEqualsMapReduceStore {

  static class Person {
    String name;

    Person(String name) { this.name = name; }

    @Override
    public boolean equals(Object o) {
        return o instanceof Person p && name.equals(p.name);
    }

    @Override
```

```
    public int hashCode() {
        return Objects.hash(name);
    }
  }

  public static void main(String[] args) {
    List<Person> people = List.of(
        new Person("Anurag"),
        new Person("Anurag"),
        new Person("Ravi")
    );

    Map<Integer, Long> result =
        people.stream()
            .distinct()
            .map(p -> p.name)                    // MAP
            .collect(Collectors.groupingBy(
                String::length,
                Collectors.counting()            // REDUCE
            ));

    System.out.println(result);
  }
}
```

---

6 distinct() immutable objects + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class DistinctImmutableMapReduceStore {
  public static void main(String[] args) {
    List<String> values = List.of("Java", "Java", "Go", "Python");

    Map<Character, Long> result =
        values.stream()
            .distinct()
            .map(String::toUpperCase)            // MAP
            .collect(Collectors.groupingBy(
                s -> s.charAt(0),
                Collectors.counting()            // REDUCE
            ));

    System.out.println(result);
  }
}
```

---

7 map() transform objects + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class MapTransformMapReduceStore {
   public static void main(String[] args) {
      List<String> names = List.of("anurag", "amit", "ravi");

      Map<Integer, Long> result =
            names.stream()
                  .map(String::toUpperCase)                    // MAP
                  .collect(Collectors.groupingBy(
                        String::length,
                        Collectors.counting()                  // REDUCE
                  ));

      System.out.println(result);
   }
}
```

---

8 map() create new objects + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class MapNewObjectMapReduceStore {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Amit", "Ravi");

      Map<Character, Long> result =
            names.stream()
                  .map(UserDTO::new)                           // MAP
                  .collect(Collectors.groupingBy(
                        dto -> dto.name.charAt(0),
                        Collectors.counting()                  // REDUCE
                  ));

      System.out.println(result);
   }
}
```

---

9 map() pure functions + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class MapPureFunctionMapReduceStore {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {
      List<Integer> numbers = List.of(2, 3, 4);

      Map<Boolean, Integer> result =
            numbers.stream()
                  .map(MapPureFunctionMapReduceStore::square) // MAP
                  .collect(Collectors.groupingBy(
                        n -> n > 10,
                        Collectors.summingInt(Integer::intValue)  // REDUCE
                  ));

      System.out.println(result);
   }
}
```

---

10 flatMap() return Stream + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FlatMapStreamMapReduceStore {
   public static void main(String[] args) {
      List<List<Integer>> lists = List.of(
            List.of(1, 2),
            List.of(3, 4)
      );

      Map<Boolean, Integer> result =
            lists.stream()
                  .flatMap(List::stream)
                  .map(n -> n * 10)                          // MAP
                  .collect(Collectors.groupingBy(
                        n -> n > 20,
                        Collectors.summingInt(Integer::intValue)  // REDUCE
                  ));

      System.out.println(result);
```

```
    }
}
```

---

11 flatMap() split data + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FlatMapSplitMapReduceStore {
    public static void main(String[] args) {
        List<String> sentences = List.of("Java Stream", "Flat Map");

        Map<Integer, Long> result =
            sentences.stream()
                .flatMap(s -> Arrays.stream(s.split(" ")))
                .map(String::toLowerCase)                    // MAP
                .collect(Collectors.groupingBy(
                    String::length,
                    Collectors.counting()                    // REDUCE
                ));

        System.out.println(result);
    }
}
```

---

12 flatMap() expand collections + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FlatMapExpandMapReduceStore {
    public static void main(String[] args) {
        Map<String, List<String>> data = Map.of(
            "A", List.of("x", "y"),
            "B", List.of("z")
        );

        Map<Integer, Long> result =
            data.values().stream()
                .flatMap(Collection::stream)
                .map(String::toUpperCase)                    // MAP
                .collect(Collectors.groupingBy(
                    String::length,
                    Collectors.counting()                    // REDUCE
                ));

        System.out.println(result);
```

```
    }
}
```

---

13 sorted() natural order + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class SortedNaturalMapReduceStore {
    public static void main(String[] args) {
        List<Integer> numbers = List.of(5, 1, 3);

        Map<Boolean, Integer> result =
            numbers.stream()
                .sorted()
                .map(n -> n * 2)                                  // MAP
                .collect(Collectors.groupingBy(
                    n -> n > 5,
                    Collectors.summingInt(Integer::intValue)  // REDUCE
                ));

        System.out.println(result);
    }
}
```

---

14 sorted() custom comparator + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class SortedCustomMapReduceStore {
    public static void main(String[] args) {
        List<String> names = List.of("Anurag", "Ravi", "Amit");

        Map<Character, Long> result =
            names.stream()
                .sorted(Comparator.reverseOrder())
                .map(String::toUpperCase)                         // MAP
                .collect(Collectors.groupingBy(
                    s -> s.charAt(0),
                    Collectors.counting()                         // REDUCE
                ));

        System.out.println(result);
    }
}
```

---

15 sorted() multi-field + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class SortedMultiFieldMapReduceStore {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {
      List<Employee> employees = List.of(
            new Employee("Anurag", 30),
            new Employee("Anurag", 25),
            new Employee("Ravi", 35)
      );

      Map<String, Integer> result =
            employees.stream()
                  .sorted(Comparator
                        .comparing((Employee e) -> e.name)
                        .thenComparingInt(e -> e.age))
                  .map(e -> e.age)                                  // MAP
                  .collect(Collectors.groupingBy(
                        age -> age >= 30 ? "SENIOR" : "JUNIOR",
                        Collectors.summingInt(Integer::intValue)  // REDUCE
                  ));

      System.out.println(result);
   }
}
```

---

16 peek() logging + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class PeekLoggingMapReduceStore {
   public static void main(String[] args) {
      Map<Boolean, Integer> result =
            List.of(1, 2, 3).stream()
```

```
                .peek(n -> System.out.println("Log: " + n))
                .map(n -> n * 10)                                   // MAP
                .collect(Collectors.groupingBy(
                        n -> n > 20,
                        Collectors.summingInt(Integer::intValue)  // REDUCE
                ));

        System.out.println(result);
    }
}
```

---

17 peek() debugging + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class PeekDebugMapReduceStore {
    public static void main(String[] args) {
        Map<Boolean, Long> result =
                List.of(10, 20, 30).stream()
                .peek(n -> {
                    if (n == 20) System.out.println("Debug hit: " + n);
                })
                .map(n -> n + 1)                                    // MAP
                .collect(Collectors.groupingBy(
                        n -> n > 20,
                        Collectors.counting()                       // REDUCE
                ));

        System.out.println(result);
    }
}
```

---

18 peek() metrics + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.*;

public class PeekMetricsMapReduceStore {
    public static void main(String[] args) {
        AtomicInteger counter = new AtomicInteger();

        Map<Boolean, Integer> result =
                List.of(5, 10, 15).stream()
                .peek(n -> counter.incrementAndGet())
                .map(n -> n * 2)                                    // MAP
```

```
            .collect(Collectors.groupingBy(
                n -> n > 20,
                Collectors.summingInt(Integer::intValue)  // REDUCE
            ));

    System.out.println("Processed: " + counter.get());
    System.out.println(result);
  }
}
```

Want to go even deeper?

I can next show:

- **Same pipelines using reducing() explicitly**
- **Parallel stream dangers in Map → Reduce**
- **Why peek() breaks referential transparency**
- **Collector internals (supplier / accumulator / combiner)**

Just say

Perfect
Below are **complete, standalone Java class examples** where **each pipeline ends with forEach() used purely for logging**.

Key rules followed:

- **One intermediate concept per class**
- **forEach() is the terminal operation**
- Logging is explicit (System.out.println)
- Java **8+**, interview-safe
- No collectors, no shortcuts

---

1 filter() boolean checks + forEach() Logging

```
import java.util.*;

public class FilterBooleanForEachLogging {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(10, 15, 20, 25);

      numbers.stream()
            .filter(n -> n % 2 == 0)                    // boolean check
            .forEach(n -> System.out.println("Even: " + n));
   }
}
```

---

2 filter() null checks + forEach() Logging

```
import java.util.*;

public class FilterNullForEachLogging {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Amit");

      names.stream()
            .filter(Objects::nonNull)
            .forEach(name -> System.out.println("Name: " + name));
   }
}
```

---

3 filter() comparisons + forEach() Logging

```
import java.util.*;
```

```
public class FilterComparisonForEachLogging {
   public static void main(String[] args) {
      List<Integer> salaries = List.of(20000, 40000, 60000);

      salaries.stream()
            .filter(s -> s >= 30000)
            .forEach(s -> System.out.println("Eligible salary: " + s));
   }
}
```

---

4 filter() method call boolean + forEach() Logging

```
import java.util.*;

public class FilterMethodForEachLogging {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {
      List<User> users = List.of(
            new User("Anurag", true),
            new User("Ravi", false)
      );

      users.stream()
            .filter(User::isActive)
            .forEach(u -> System.out.println("Active user: " + u.name));
   }
}
```

---

5 distinct() equals/hashCode + forEach() Logging

```
import java.util.*;

public class DistinctEqualsForEachLogging {
```

```
    static class Person {
        String name;

        Person(String name) {
            this.name = name;
        }

        @Override
        public boolean equals(Object o) {
            return o instanceof Person p && name.equals(p.name);
        }

        @Override
        public int hashCode() {
            return Objects.hash(name);
        }
    }

    public static void main(String[] args) {
        List<Person> people = List.of(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        );

        people.stream()
                .distinct()
                .forEach(p -> System.out.println("Unique person: " + p.name));
    }
}
```

---

6 distinct() immutable objects + forEach() Logging

```
import java.util.*;

public class DistinctImmutableForEachLogging {
    public static void main(String[] args) {
        List<String> languages = List.of("Java", "Java", "Python");

        languages.stream()
                .distinct()
                .forEach(lang -> System.out.println("Language: " + lang));
    }
}
```

---

7 map() transform objects + forEach() Logging

```
import java.util.*;

public class MapTransformForEachLogging {
   public static void main(String[] args) {
      List<String> names = List.of("anurag", "amit");

      names.stream()
            .map(String::toUpperCase)
            .forEach(name -> System.out.println("Upper: " + name));
   }
}
```

---

8 map() create new objects + forEach() Logging

```
import java.util.*;

public class MapNewObjectForEachLogging {

   static class UserDTO {
      String name;

      UserDTO(String name) {
         this.name = name;
      }
   }

   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Amit");

      names.stream()
            .map(UserDTO::new)
            .forEach(dto -> System.out.println("DTO created: " + dto.name));
   }
}
```

---

9 map() pure functions + forEach() Logging

```
import java.util.*;

public class MapPureFunctionForEachLogging {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {
      List<Integer> numbers = List.of(2, 3, 4);
```

```
        numbers.stream()
            .map(MapPureFunctionForEachLogging::square)
            .forEach(n -> System.out.println("Square: " + n));
    }
}
```

---

10 flatMap() return Stream + forEach() Logging

```
import java.util.*;

public class FlatMapStreamForEachLogging {
    public static void main(String[] args) {
        List<List<Integer>> lists = List.of(
            List.of(1, 2),
            List.of(3, 4)
        );

        lists.stream()
            .flatMap(List::stream)
            .forEach(n -> System.out.println("Value: " + n));
    }
}
```

---

11 flatMap() split data + forEach() Logging

```
import java.util.*;

public class FlatMapSplitForEachLogging {
    public static void main(String[] args) {
        List<String> sentences = List.of("Java Stream", "Flat Map");

        sentences.stream()
            .flatMap(s -> Arrays.stream(s.split(" ")))
            .forEach(word -> System.out.println("Word: " + word));
    }
}
```

---

12 flatMap() expand collections + forEach() Logging

```
import java.util.*;

public class FlatMapExpandForEachLogging {
    public static void main(String[] args) {
        Map<String, List<String>> data = Map.of(
            "A", List.of("x", "y"),
```

```
            "B", List.of("z")
        );

        data.values().stream()
            .flatMap(Collection::stream)
            .forEach(v -> System.out.println("Value: " + v));
    }
}
```

---

13 sorted() natural ordering + forEach() Logging

```
import java.util.*;

public class SortedNaturalForEachLogging {
    public static void main(String[] args) {
        List<Integer> numbers = List.of(5, 1, 3);

        numbers.stream()
            .sorted()
            .forEach(n -> System.out.println("Sorted: " + n));
    }
}
```

---

14 sorted() custom comparator + forEach() Logging

```
import java.util.*;

public class SortedCustomForEachLogging {
    public static void main(String[] args) {
        List<String> names = List.of("Anurag", "Ravi", "Amit");

        names.stream()
            .sorted(Comparator.reverseOrder())
            .forEach(name -> System.out.println("Name: " + name));
    }
}
```

---

15 sorted() multi-field + forEach() Logging

```
import java.util.*;

public class SortedMultiFieldForEachLogging {

    static class Employee {
        String name;
        int age;
```

```
    Employee(String name, int age) {
        this.name = name;
        this.age = age;
    }
  }

  public static void main(String[] args) {
    List<Employee> employees = List.of(
          new Employee("Anurag", 30),
          new Employee("Anurag", 25),
          new Employee("Ravi", 35)
    );

    employees.stream()
          .sorted(Comparator
                .comparing((Employee e) -> e.name)
                .thenComparingInt(e -> e.age))
          .forEach(e ->
                System.out.println(e.name + " - " + e.age));
  }
}
```

---

16 peek() logging + forEach() Logging

```
import java.util.*;

public class PeekLoggingForEachLogging {
  public static void main(String[] args) {
    List.of(1, 2, 3).stream()
          .peek(n -> System.out.println("Peek log: " + n))
          .forEach(n -> System.out.println("Final: " + n));
  }
}
```

---

17 peek() debugging + forEach() Logging

```
import java.util.*;

public class PeekDebugForEachLogging {
  public static void main(String[] args) {
    List.of(10, 20, 30).stream()
          .peek(n -> {
            if (n == 20) {
                System.out.println("Debug breakpoint hit: " + n);
            }
          })
```

```
            .forEach(n -> System.out.println("Processed: " + n));
    }
}
```

---

18 peek() metrics + forEach() Logging

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsForEachLogging {
    public static void main(String[] args) {
        AtomicInteger counter = new AtomicInteger();

        List.of(5, 10, 15).stream()
            .peek(n -> counter.incrementAndGet())
            .forEach(n -> System.out.println("Value: " + n));

        System.out.println("Total processed: " + counter.get());
    }
}
```

If you want next

I can show:

- **Why forEach() breaks parallel stream determinism**
- **forEach() vs forEachOrdered()**
- **When logging inside streams is a bad idea**
- **Same examples rewritten with proper logging frameworks (SLF4J)**

Just say

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **One clear intermediate operation** is demonstrated
- **forEach() is the terminal operation**
- **Purpose = Printing output (not logging, not collecting)**
- Java **8+ compatible**
- Clean, interview-ready, minimal noise

---

1 filter() boolean checks + forEach() Printing

```
import java.util.*;

public class FilterBooleanForEachPrint {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(10, 15, 20, 25);

      numbers.stream()
            .filter(n -> n % 2 == 0)
            .forEach(System.out::println);
   }
}
```

---

2 filter() null checks + forEach() Printing

```
import java.util.*;

public class FilterNullForEachPrint {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Amit");

      names.stream()
            .filter(Objects::nonNull)
            .forEach(System.out::println);
   }
}
```

---

3 filter() comparisons + forEach() Printing

```
import java.util.*;

public class FilterComparisonForEachPrint {
   public static void main(String[] args) {
      List<Integer> salaries = List.of(20000, 40000, 60000);
```

```
    salaries.stream()
        .filter(s -> s >= 30000)
        .forEach(System.out::println);
  }
}
```

---

4 filter() method call returning boolean + forEach() Printing

```
import java.util.*;

public class FilterMethodForEachPrint {

  static class User {
    String name;
    boolean active;

    User(String name, boolean active) {
      this.name = name;
      this.active = active;
    }

    boolean isActive() {
      return active;
    }
  }

  public static void main(String[] args) {
    List<User> users = List.of(
        new User("Anurag", true),
        new User("Ravi", false)
    );

    users.stream()
        .filter(User::isActive)
        .forEach(u -> System.out.println(u.name));
  }
}
```

---

5 distinct() rely on equals() / hashCode() + forEach() Printing

```
import java.util.*;

public class DistinctEqualsForEachPrint {

  static class Person {
    String name;
```

```
    Person(String name) { this.name = name; }

    @Override
    public boolean equals(Object o) {
      return o instanceof Person p && name.equals(p.name);
    }

    @Override
    public int hashCode() {
      return Objects.hash(name);
    }
  }

  public static void main(String[] args) {
    List<Person> people = List.of(
        new Person("Anurag"),
        new Person("Anurag"),
        new Person("Ravi")
    );

    people.stream()
        .distinct()
        .forEach(p -> System.out.println(p.name));
  }
}
```

---

6 distinct() immutable objects + forEach() Printing

```
import java.util.*;

public class DistinctImmutableForEachPrint {
  public static void main(String[] args) {
    List<String> languages = List.of("Java", "Java", "Python");

    languages.stream()
        .distinct()
        .forEach(System.out::println);
  }
}
```

---

7 map() transform objects + forEach() Printing

```
import java.util.*;

public class MapTransformForEachPrint {
  public static void main(String[] args) {
    List<String> names = List.of("anurag", "amit");
```

```
        names.stream()
            .map(String::toUpperCase)
            .forEach(System.out::println);
    }
}
```

---

8 map() create new objects + forEach() Printing

```
import java.util.*;

public class MapNewObjectForEachPrint {

    static class UserDTO {
        String name;
        UserDTO(String name) { this.name = name; }
    }

    public static void main(String[] args) {
        List<String> names = List.of("Anurag", "Amit");

        names.stream()
            .map(UserDTO::new)
            .forEach(dto -> System.out.println(dto.name));
    }
}
```

---

9 map() call pure functions + forEach() Printing

```
import java.util.*;

public class MapPureFunctionForEachPrint {

    static int square(int n) {
        return n * n;
    }

    public static void main(String[] args) {
        List<Integer> numbers = List.of(2, 3, 4);

        numbers.stream()
            .map(MapPureFunctionForEachPrint::square)
            .forEach(System.out::println);
    }
}
```

---

10 flatMap() return Stream + forEach() Printing

```
import java.util.*;

public class FlatMapStreamForEachPrint {
   public static void main(String[] args) {
      List<List<Integer>> lists = List.of(
            List.of(1, 2),
            List.of(3, 4)
      );

      lists.stream()
            .flatMap(List::stream)
            .forEach(System.out::println);
   }
}
```

---

11 flatMap() split data + forEach() Printing

```
import java.util.*;

public class FlatMapSplitForEachPrint {
   public static void main(String[] args) {
      List<String> sentences = List.of("Java Stream", "Flat Map");

      sentences.stream()
            .flatMap(s -> Arrays.stream(s.split(" ")))
            .forEach(System.out::println);
   }
}
```

---

12 flatMap() expand collections + forEach() Printing

```
import java.util.*;

public class FlatMapExpandForEachPrint {
   public static void main(String[] args) {
      Map<String, List<String>> data = Map.of(
            "A", List.of("x", "y"),
            "B", List.of("z")
      );

      data.values().stream()
            .flatMap(Collection::stream)
            .forEach(System.out::println);
   }
}
```

---

13 sorted() natural ordering + forEach() Printing

```
import java.util.*;

public class SortedNaturalForEachPrint {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(5, 1, 3);

      numbers.stream()
            .sorted()
            .forEach(System.out::println);
   }
}
```

---

14 sorted() custom comparator + forEach() Printing

```
import java.util.*;

public class SortedCustomForEachPrint {
   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Ravi", "Amit");

      names.stream()
            .sorted(Comparator.reverseOrder())
            .forEach(System.out::println);
   }
}
```

---

15 sorted() multi-field compare + forEach() Printing

```
import java.util.*;

public class SortedMultiFieldForEachPrint {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {
```

```
    List<Employee> employees = List.of(
        new Employee("Anurag", 30),
        new Employee("Anurag", 25),
        new Employee("Ravi", 35)
    );

    employees.stream()
        .sorted(Comparator
            .comparing((Employee e) -> e.name)
            .thenComparingInt(e -> e.age))
        .forEach(e -> System.out.println(e.name + " " + e.age));
  }
}
```

---

16 peek() logging + forEach() Printing

```
import java.util.*;

public class PeekLoggingForEachPrint {
  public static void main(String[] args) {
    List.of(1, 2, 3).stream()
        .peek(n -> System.out.println("Peek: " + n))
        .forEach(System.out::println);
  }
}
```

---

17 peek() debugging + forEach() Printing

```
import java.util.*;

public class PeekDebugForEachPrint {
  public static void main(String[] args) {
    List.of(10, 20, 30).stream()
        .peek(n -> {
          if (n == 20) System.out.println("Debug hit");
        })
        .forEach(System.out::println);
  }
}
```

---

18 peek() metrics + forEach() Printing

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsForEachPrint {
```

```
    public static void main(String[] args) {
        AtomicInteger counter = new AtomicInteger();

        List.of(5, 10, 15).stream()
                .peek(n -> counter.incrementAndGet())
                .forEach(System.out::println);

        System.out.println("Total processed: " + counter.get());
    }
}
```

✅ **You've now covered ALL major Stream patterns**

If you want next, I can give:

- **forEach vs forEachOrdered**
- **Parallel stream versions**
- **Illegal vs legal side-effects**
- **One master table mapping operation → intent → misuse**

Just tell me.

Below are **complete, standalone Java class examples** showing **side-effects with forEach()** where the stream **updates an external system** (simulated as DB / cache / message queue / API).

 Important concept reminder (intentional here):
Using forEach() to update external systems is a **side-effect** and must be done **only at the terminal operation**. These examples are **educational and realistic**, not advocating misuse.

All examples:

- Java **8+**
- One intermediate concept per class
- External system = simulated service (ExternalSystem.update(...))

Shared helper (used conceptually in all examples)

```
class ExternalSystem {
   static void update(String message) {
      System.out.println("External update → " + message);
   }
}
```

(Each class below assumes this helper exists in the same file or package.)

---

1 filter() boolean checks + forEach() Updating external systems

```
import java.util.*;

public class FilterBooleanUpdateExternal {
   public static void main(String[] args) {
      List<Integer> orders = List.of(100, 150, 200);

      orders.stream()
            .filter(o -> o >= 150)
            .forEach(o -> ExternalSystem.update("Approved order: " + o));
   }
}
```

---

2 filter() null checks + forEach() Updating external systems

```
import java.util.*;

public class FilterNullUpdateExternal {
   public static void main(String[] args) {
```

```
    List<String> users = Arrays.asList("Anurag", null, "Amit");

    users.stream()
        .filter(Objects::nonNull)
        .forEach(u -> ExternalSystem.update("User synced: " + u));
  }
}
```

---

3 filter() comparisons + forEach() Updating external systems

```
import java.util.*;

public class FilterComparisonUpdateExternal {
  public static void main(String[] args) {
    List<Integer> balances = List.of(500, 2000, 7000);

    balances.stream()
        .filter(b -> b > 1000)
        .forEach(b -> ExternalSystem.update("Premium balance: " + b));
  }
}
```

---

4 filter() method call returning boolean + forEach() Updating external systems

```
import java.util.*;

public class FilterMethodUpdateExternal {

  static class Account {
    String id;
    boolean active;

    Account(String id, boolean active) {
      this.id = id;
      this.active = active;
    }

    boolean isActive() {
      return active;
    }
  }

  public static void main(String[] args) {
    List<Account> accounts = List.of(
        new Account("A1", true),
        new Account("A2", false)
    );
```

```
    accounts.stream()
        .filter(Account::isActive)
        .forEach(a -> ExternalSystem.update("Activated account: " + a.id));
  }
}
```

---

5 distinct() equals/hashCode + forEach() Updating external systems

```
import java.util.*;

public class DistinctEqualsUpdateExternal {

  static class Customer {
    String email;

    Customer(String email) { this.email = email; }

    @Override
    public boolean equals(Object o) {
      return o instanceof Customer c && email.equals(c.email);
    }

    @Override
    public int hashCode() {
      return Objects.hash(email);
    }
  }

  public static void main(String[] args) {
    List<Customer> customers = List.of(
        new Customer("a@mail.com"),
        new Customer("a@mail.com"),
        new Customer("b@mail.com")
    );

    customers.stream()
        .distinct()
        .forEach(c -> ExternalSystem.update("Customer saved: " + c.email));
  }
}
```

---

6 distinct() immutable objects + forEach() Updating external systems

```
import java.util.*;

public class DistinctImmutableUpdateExternal {
```

```
    public static void main(String[] args) {
        List<String> tags = List.of("java", "java", "spring");

        tags.stream()
                .distinct()
                .forEach(t -> ExternalSystem.update("Tag indexed: " + t));
    }
}
```

---

7 map() transform objects + forEach() Updating external systems

```
import java.util.*;

public class MapTransformUpdateExternal {
    public static void main(String[] args) {
        List<String> names = List.of("anurag", "amit");

        names.stream()
                .map(String::toUpperCase)
                .forEach(n -> ExternalSystem.update("Normalized name: " + n));
    }
}
```

---

8 map() create new objects + forEach() Updating external systems

```
import java.util.*;

public class MapNewObjectUpdateExternal {

    static class UserDTO {
        String name;
        UserDTO(String name) { this.name = name; }
    }

    public static void main(String[] args) {
        List<String> names = List.of("Anurag", "Amit");

        names.stream()
                .map(UserDTO::new)
                .forEach(dto -> ExternalSystem.update("DTO persisted: " + dto.name));
    }
}
```

---

9 map() call pure functions + forEach() Updating external systems

```
import java.util.*;
```

```
public class MapPureFunctionUpdateExternal {

   static int calculateScore(int n) {
      return n * 10;
   }

   public static void main(String[] args) {
      List<Integer> inputs = List.of(1, 2, 3);

      inputs.stream()
            .map(MapPureFunctionUpdateExternal::calculateScore)
            .forEach(score -> ExternalSystem.update("Score sent: " + score));
   }
}
```

---

10 flatMap() return Stream + forEach() Updating external systems

```
import java.util.*;

public class FlatMapStreamUpdateExternal {
   public static void main(String[] args) {
      List<List<String>> batches = List.of(
            List.of("A", "B"),
            List.of("C")
      );

      batches.stream()
            .flatMap(List::stream)
            .forEach(v -> ExternalSystem.update("Batch item: " + v));
   }
}
```

---

11 flatMap() split data + forEach() Updating external systems

```
import java.util.*;

public class FlatMapSplitUpdateExternal {
   public static void main(String[] args) {
      List<String> lines = List.of("Java Stream", "Flat Map");

      lines.stream()
            .flatMap(l -> Arrays.stream(l.split(" ")))
            .forEach(word -> ExternalSystem.update("Word indexed: " + word));
   }
}
```

---

12 flatMap() expand collections + forEach() Updating external systems

```
import java.util.*;

public class FlatMapExpandUpdateExternal {
   public static void main(String[] args) {
      Map<String, List<String>> data = Map.of(
            "A", List.of("1", "2"),
            "B", List.of("3")
      );

      data.values().stream()
            .flatMap(Collection::stream)
            .forEach(v -> ExternalSystem.update("Value synced: " + v));
   }
}
```

---

13 sorted() natural ordering + forEach() Updating external systems

```
import java.util.*;

public class SortedNaturalUpdateExternal {
   public static void main(String[] args) {
      List<Integer> priorities = List.of(3, 1, 2);

      priorities.stream()
            .sorted()
            .forEach(p -> ExternalSystem.update("Processed priority: " + p));
   }
}
```

---

14 sorted() custom comparator + forEach() Updating external systems

```
import java.util.*;

public class SortedCustomUpdateExternal {
   public static void main(String[] args) {
      List<String> jobs = List.of("LOW", "HIGH", "MEDIUM");

      jobs.stream()
            .sorted(Comparator.reverseOrder())
            .forEach(j -> ExternalSystem.update("Job executed: " + j));
   }
}
```

---

15 sorted() multi-field compare + forEach() Updating external systems

```
import java.util.*;

public class SortedMultiFieldUpdateExternal {

   static class Task {
      String name;
      int priority;

      Task(String name, int priority) {
         this.name = name;
         this.priority = priority;
      }
   }

   public static void main(String[] args) {
      List<Task> tasks = List.of(
            new Task("Deploy", 2),
            new Task("Build", 1),
            new Task("Test", 2)
      );

      tasks.stream()
            .sorted(Comparator
                  .comparing((Task t) -> t.priority)
                  .thenComparing(t -> t.name))
            .forEach(t ->
                  ExternalSystem.update("Task executed: " + t.name));
   }
}
```

---

16 peek() logging + forEach() Updating external systems

```
import java.util.*;

public class PeekLoggingUpdateExternal {
   public static void main(String[] args) {
      List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Peek log: " + n))
            .forEach(n -> ExternalSystem.update("Final update: " + n));
   }
}
```

---

17 peek() debugging + forEach() Updating external systems

```
import java.util.*;

public class PeekDebugUpdateExternal {
```

```
    public static void main(String[] args) {
        List.of(10, 20, 30).stream()
            .peek(n -> {
                if (n == 20) System.out.println("Debug breakpoint hit");
            })
            .forEach(n -> ExternalSystem.update("Processed value: " + n));
    }
}
```

---

18 peek() metrics + forEach() Updating external systems

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsUpdateExternal {
    public static void main(String[] args) {
        AtomicInteger counter = new AtomicInteger();

        List.of(5, 10, 15).stream()
            .peek(n -> counter.incrementAndGet())
            .forEach(n -> ExternalSystem.update("Metric sent: " + n));

        ExternalSystem.update("Total processed: " + counter.get());
    }
}
```

⚠ Final Expert Note (very important)

- These patterns **must NOT** be used with **parallel streams**
- External updates must be **idempotent**
- Prefer **batching + collectors** when possible

If you want next:

- ❌ **Anti-patterns with forEach()**
- ✅ **Safe alternatives**
- ⚡ **Parallel stream failure demos**
- □ **Transactional stream design**

Just say the word.

Below are **complete, standalone Java class examples** for **every requested Stream combination**, where:

- **One primary intermediate operation** is demonstrated
- **reduce() is the terminal operation**
- **Goal = combine values** (sum, concat, merge)
- Java **8+ compatible**
- Clean, deterministic, interview-ready
- No collectors, no forEach

 Mental model
reduce(identity, accumulator) ⇒ **Combine stream elements into ONE result**

---

1 filter() boolean checks + reduce() Combine values

```
import java.util.*;

public class FilterBooleanReduce {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(10, 15, 20, 25);

      int sum =
            numbers.stream()
                  .filter(n -> n % 2 == 0)
                  .reduce(0, Integer::sum);

      System.out.println(sum);
   }
}
```

---

2 filter() null checks + reduce() Combine values

```
import java.util.*;

public class FilterNullReduce {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Amit");

      String combined =
            names.stream()
                  .filter(Objects::nonNull)
                  .reduce("", (a, b) -> a + b);

      System.out.println(combined);
   }
}
```

---

3 filter() comparisons + reduce() Combine values

```
import java.util.*;

public class FilterComparisonReduce {
   public static void main(String[] args) {
      List<Integer> salaries = List.of(20000, 40000, 60000);

      int total =
            salaries.stream()
                  .filter(s -> s >= 30000)
                  .reduce(0, Integer::sum);

      System.out.println(total);
   }
}
```

---

4 filter() method call boolean + reduce() Combine values

```
import java.util.*;

public class FilterMethodReduce {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {
      List<User> users = List.of(
            new User("Anurag", true),
            new User("Amit", true),
            new User("Ravi", false)
      );

      int count =
            users.stream()
```

```
                .filter(User::isActive)
                .map(u -> 1)
                .reduce(0, Integer::sum);

    System.out.println(count);
  }
}
```

---

5 distinct() equals/hashCode + reduce() Combine values

```
import java.util.*;

public class DistinctEqualsReduce {

  static class Person {
    String name;

    Person(String name) { this.name = name; }

    @Override
    public boolean equals(Object o) {
      return o instanceof Person p && name.equals(p.name);
    }

    @Override
    public int hashCode() {
      return Objects.hash(name);
    }
  }

  public static void main(String[] args) {
    List<Person> people = List.of(
        new Person("Anurag"),
        new Person("Anurag"),
        new Person("Ravi")
    );

    int totalLength =
        people.stream()
            .distinct()
            .map(p -> p.name.length())
            .reduce(0, Integer::sum);

    System.out.println(totalLength);
  }
}
```

---

6 distinct() immutable objects + reduce() Combine values

```
import java.util.*;

public class DistinctImmutableReduce {
   public static void main(String[] args) {
      List<String> values = List.of("Java", "Java", "Go");

      int lengthSum =
            values.stream()
                  .distinct()
                  .map(String::length)
                  .reduce(0, Integer::sum);

      System.out.println(lengthSum);
   }
}
```

---

7 map() transform objects + reduce() Combine values

```
import java.util.*;

public class MapTransformReduce {
   public static void main(String[] args) {
      List<String> names = List.of("anurag", "amit");

      int totalChars =
            names.stream()
                  .map(String::toUpperCase)
                  .map(String::length)
                  .reduce(0, Integer::sum);

      System.out.println(totalChars);
   }
}
```

---

8 map() create new objects + reduce() Combine values

```
import java.util.*;

public class MapNewObjectReduce {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }
```

```
    public static void main(String[] args) {
        List<String> names = List.of("Anurag", "Amit");

        int count =
                names.stream()
                        .map(UserDTO::new)
                        .map(dto -> 1)
                        .reduce(0, Integer::sum);

        System.out.println(count);
    }
}
```

---

9 map() pure functions + reduce() Combine values

```
import java.util.*;

public class MapPureFunctionReduce {

    static int square(int n) {
        return n * n;
    }

    public static void main(String[] args) {
        List<Integer> numbers = List.of(2, 3, 4);

        int sumOfSquares =
                numbers.stream()
                        .map(MapPureFunctionReduce::square)
                        .reduce(0, Integer::sum);

        System.out.println(sumOfSquares);
    }
}
```

---

10 flatMap() return Stream + reduce() Combine values

```
import java.util.*;

public class FlatMapStreamReduce {
    public static void main(String[] args) {
        List<List<Integer>> lists = List.of(
                List.of(1, 2),
                List.of(3, 4)
        );

        int total =
```

```
        lists.stream()
            .flatMap(List::stream)
            .reduce(0, Integer::sum);

    System.out.println(total);
  }
}
```

---

11 flatMap() split data + reduce() Combine values

```
import java.util.*;

public class FlatMapSplitReduce {
  public static void main(String[] args) {
    List<String> sentences = List.of("Java Stream", "Flat Map");

    int wordCount =
        sentences.stream()
            .flatMap(s -> Arrays.stream(s.split(" ")))
            .map(w -> 1)
            .reduce(0, Integer::sum);

    System.out.println(wordCount);
  }
}
```

---

12 flatMap() expand collections + reduce() Combine values

```
import java.util.*;

public class FlatMapExpandReduce {
  public static void main(String[] args) {
    Map<String, List<Integer>> data = Map.of(
        "A", List.of(1, 2),
        "B", List.of(3)
    );

    int sum =
        data.values().stream()
            .flatMap(Collection::stream)
            .reduce(0, Integer::sum);

    System.out.println(sum);
  }
}
```

---

13 sorted() natural ordering + reduce() Combine values

```
import java.util.*;

public class SortedNaturalReduce {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(5, 1, 3);

      int total =
            numbers.stream()
                  .sorted()
                  .reduce(0, Integer::sum);

      System.out.println(total);
   }
}
```

---

14 sorted() custom comparator + reduce() Combine values

```
import java.util.*;

public class SortedCustomReduce {
   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Ravi", "Amit");

      String combined =
            names.stream()
                  .sorted(Comparator.reverseOrder())
                  .reduce("", (a, b) -> a + b);

      System.out.println(combined);
   }
}
```

---

15 sorted() multi-field compare + reduce() Combine values

```
import java.util.*;

public class SortedMultiFieldReduce {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
```

```
    }
  }

  public static void main(String[] args) {
    List<Employee> employees = List.of(
        new Employee("Anurag", 30),
        new Employee("Anurag", 25),
        new Employee("Ravi", 35)
    );

    int ageSum =
        employees.stream()
            .sorted(Comparator
                .comparing((Employee e) -> e.name)
                .thenComparingInt(e -> e.age))
            .map(e -> e.age)
            .reduce(0, Integer::sum);

    System.out.println(ageSum);
  }
}
```

---

16 peek() logging + reduce() Combine values

```
import java.util.*;

public class PeekLoggingReduce {
  public static void main(String[] args) {
    int sum =
        List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Peek: " + n))
            .reduce(0, Integer::sum);

    System.out.println(sum);
  }
}
```

---

17 peek() debugging + reduce() Combine values

```
import java.util.*;

public class PeekDebugReduce {
  public static void main(String[] args) {
    int sum =
        List.of(10, 20, 30).stream()
            .peek(n -> {
              if (n == 20) System.out.println("Debug hit");
```

```
                })
                .reduce(0, Integer::sum);

        System.out.println(sum);
    }
}
```

---

18 peek() metrics + reduce() Combine values

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsReduce {
    public static void main(String[] args) {
        AtomicInteger counter = new AtomicInteger();

        int sum =
            List.of(5, 10, 15).stream()
                .peek(n -> counter.incrementAndGet())
                .reduce(0, Integer::sum);

        System.out.println("Sum: " + sum);
        System.out.println("Processed: " + counter.get());
    }
}
```

 Expert Notes (very important)

- reduce() **must be associative**
- Prefer **immutable accumulations**
- Never mutate external state inside reduce
- For grouping → prefer collect()

If you want next:

- **reduce(identity, accumulator, combiner)**
- **Why reduce is dangerous with objects**
- **Parallel reduce correctness**
- **Collector vs reduce decision table**

Just say the word

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where the **terminal operation is reduce()** and it demonstrates **aggregation**:

- **SUM** (numeric aggregation)
- **MAX** (numeric aggregation)
- **CONCAT** (string aggregation)

Each class:

- Java **8+**
- One **main intermediate focus**
- Uses **only reduce()** (no collectors)
- Clear, interview-safe, deterministic

---

1 filter() boolean checks + reduce() Aggregate

```
import java.util.*;

public class FilterBooleanReduceAggregate {
   public static void main(String[] args) {
      List<Integer> nums = List.of(10, 15, 20);

      int sum = nums.stream()
            .filter(n -> n % 2 == 0)
            .reduce(0, Integer::sum);

      int max = nums.stream()
            .filter(n -> n % 2 == 0)
            .reduce(Integer::max)
            .orElse(0);

      String concat = nums.stream()
            .filter(n -> n % 2 == 0)
            .map(String::valueOf)
            .reduce("", String::concat);

      System.out.println(sum + " | " + max + " | " + concat);
   }
}
```

---

2 filter() null checks + reduce() Aggregate

```
import java.util.*;

public class FilterNullReduceAggregate {
   public static void main(String[] args) {
```

```
    List<String> names = Arrays.asList("Anurag", null, "Amit");

    String concat = names.stream()
        .filter(Objects::nonNull)
        .reduce("", String::concat);

    int maxLength = names.stream()
        .filter(Objects::nonNull)
        .map(String::length)
        .reduce(Integer::max)
        .orElse(0);

    System.out.println(concat + " | " + maxLength);
  }
}
```

---

3 filter() comparisons + reduce() Aggregate

```
import java.util.*;

public class FilterComparisonReduceAggregate {
  public static void main(String[] args) {
    List<Integer> salaries = List.of(20000, 40000, 60000);

    int sum = salaries.stream()
        .filter(s -> s >= 30000)
        .reduce(0, Integer::sum);

    int max = salaries.stream()
        .filter(s -> s >= 30000)
        .reduce(Integer::max)
        .orElse(0);

    System.out.println(sum + " | " + max);
  }
}
```

---

4 filter() method call boolean + reduce() Aggregate

```
import java.util.*;

public class FilterMethodReduceAggregate {

  static class User {
    String name;
    boolean active;
```

```
    User(String name, boolean active) {
        this.name = name;
        this.active = active;
    }

    boolean isActive() { return active; }
  }

  public static void main(String[] args) {
    List<User> users = List.of(
            new User("Anurag", true),
            new User("Amit", true),
            new User("Ravi", false)
    );

    int count = users.stream()
            .filter(User::isActive)
            .map(u -> 1)
            .reduce(0, Integer::sum);

    String concat = users.stream()
            .filter(User::isActive)
            .map(u -> u.name)
            .reduce("", String::concat);

    System.out.println(count + " | " + concat);
  }
}
```

---

5 distinct() equals/hashCode + reduce() Aggregate

```
import java.util.*;

public class DistinctEqualsReduceAggregate {

  static class Person {
    String name;
    Person(String name) { this.name = name; }

    @Override
    public boolean equals(Object o) {
        return o instanceof Person p && name.equals(p.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name);
    }
```

```
    }

    public static void main(String[] args) {
        List<Person> people = List.of(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        );

        String concat = people.stream()
                .distinct()
                .map(p -> p.name)
                .reduce("", String::concat);

        System.out.println(concat);
    }
}
```

---

6 distinct() immutable objects + reduce() Aggregate

```
import java.util.*;

public class DistinctImmutableReduceAggregate {
    public static void main(String[] args) {
        List<String> langs = List.of("Java", "Java", "Go");

        String concat = langs.stream()
                .distinct()
                .reduce("", String::concat);

        int maxLen = langs.stream()
                .distinct()
                .map(String::length)
                .reduce(Integer::max)
                .orElse(0);

        System.out.println(concat + " | " + maxLen);
    }
}
```

---

7 map() transform objects + reduce() Aggregate

```
import java.util.*;

public class MapTransformReduceAggregate {
    public static void main(String[] args) {
        List<String> names = List.of("anurag", "amit");
```

```
    String concat = names.stream()
        .map(String::toUpperCase)
        .reduce("", String::concat);

    int maxLen = names.stream()
        .map(String::toUpperCase)
        .map(String::length)
        .reduce(Integer::max)
        .orElse(0);

    System.out.println(concat + " | " + maxLen);
  }
}
```

---

8 map() create new objects + reduce() Aggregate

```
import java.util.*;

public class MapNewObjectReduceAggregate {

  static class UserDTO {
    String name;
    UserDTO(String name) { this.name = name; }
  }

  public static void main(String[] args) {
    List<String> names = List.of("Anurag", "Amit");

    String concat = names.stream()
        .map(UserDTO::new)
        .map(dto -> dto.name)
        .reduce("", String::concat);

    System.out.println(concat);
  }
}
```

---

9 map() pure functions + reduce() Aggregate

```
import java.util.*;

public class MapPureFunctionReduceAggregate {

  static int square(int n) { return n * n; }

  public static void main(String[] args) {
```

```
    List<Integer> nums = List.of(2, 3, 4);

    int sum = nums.stream()
        .map(MapPureFunctionReduceAggregate::square)
        .reduce(0, Integer::sum);

    int max = nums.stream()
        .map(MapPureFunctionReduceAggregate::square)
        .reduce(Integer::max)
        .orElse(0);

    System.out.println(sum + " | " + max);
  }
}
```

---

10 flatMap() return Stream + reduce() Aggregate

```
import java.util.*;

public class FlatMapStreamReduceAggregate {
  public static void main(String[] args) {
    List<List<Integer>> lists = List.of(
        List.of(1, 2),
        List.of(3, 4)
    );

    int sum = lists.stream()
        .flatMap(List::stream)
        .reduce(0, Integer::sum);

    System.out.println(sum);
  }
}
```

---

11 flatMap() split data + reduce() Aggregate

```
import java.util.*;

public class FlatMapSplitReduceAggregate {
  public static void main(String[] args) {
    List<String> lines = List.of("Java Stream", "Flat Map");

    String concat = lines.stream()
        .flatMap(l -> Arrays.stream(l.split(" ")))
        .reduce("", String::concat);

    System.out.println(concat);
```

```
    }
}
```

---

12 flatMap() expand collections + reduce() Aggregate

```
import java.util.*;

public class FlatMapExpandReduceAggregate {
    public static void main(String[] args) {
        Map<String, List<Integer>> map = Map.of(
                "A", List.of(1, 2),
                "B", List.of(3)
        );

        int sum = map.values().stream()
                .flatMap(Collection::stream)
                .reduce(0, Integer::sum);

        System.out.println(sum);
    }
}
```

---

13 sorted() natural ordering + reduce() Aggregate

```
import java.util.*;

public class SortedNaturalReduceAggregate {
    public static void main(String[] args) {
        List<Integer> nums = List.of(5, 1, 3);

        int max = nums.stream()
                .sorted()
                .reduce(Integer::max)
                .orElse(0);

        System.out.println(max);
    }
}
```

---

14 sorted() custom comparator + reduce() Aggregate

```
import java.util.*;

public class SortedCustomReduceAggregate {
    public static void main(String[] args) {
        List<String> names = List.of("Anurag", "Ravi", "Amit");
```

```
    String concat = names.stream()
        .sorted(Comparator.reverseOrder())
        .reduce("", String::concat);

    System.out.println(concat);
  }
}
```

---

15 sorted() multi-field compare + reduce() Aggregate

```
import java.util.*;

public class SortedMultiFieldReduceAggregate {

  static class Employee {
    String name;
    int age;
    Employee(String name, int age) {
      this.name = name; this.age = age;
    }
  }

  public static void main(String[] args) {
    List<Employee> emps = List.of(
        new Employee("Anurag", 30),
        new Employee("Anurag", 25),
        new Employee("Ravi", 35)
    );

    int maxAge = emps.stream()
        .sorted(Comparator
            .comparing((Employee e) -> e.name)
            .thenComparingInt(e -> e.age))
        .map(e -> e.age)
        .reduce(Integer::max)
        .orElse(0);

    System.out.println(maxAge);
  }
}
```

---

16 peek() logging + reduce() Aggregate

```
import java.util.*;

public class PeekLoggingReduceAggregate {
```

```
    public static void main(String[] args) {
        int sum = List.of(1, 2, 3).stream()
                .peek(n -> System.out.println("Peek: " + n))
                .reduce(0, Integer::sum);

        System.out.println(sum);
    }
}
```

---

17 peek() debugging + reduce() Aggregate

```
import java.util.*;

public class PeekDebugReduceAggregate {
    public static void main(String[] args) {
        int max = List.of(10, 20, 30).stream()
                .peek(n -> {
                    if (n == 20) System.out.println("Debug hit");
                })
                .reduce(Integer::max)
                .orElse(0);

        System.out.println(max);
    }
}
```

---

18 peek() metrics + reduce() Aggregate

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsReduceAggregate {
    public static void main(String[] args) {
        AtomicInteger counter = new AtomicInteger();

        int sum = List.of(5, 10, 15).stream()
                .peek(n -> counter.incrementAndGet())
                .reduce(0, Integer::sum);

        System.out.println("Sum: " + sum);
        System.out.println("Processed: " + counter.get());
    }
}
```

⚠ Final Expert Takeaways

- reduce() must be **associative + stateless**
- Avoid **mutable objects** in reduce
- Prefer collect() for grouping
- peek() is **observational only**

If you want, next I can give:

- 🧮 reduce(identity, accumulator, combiner)
- 🧵 Parallel reduce correctness proofs
- 📊 Reduce vs Collect decision matrix

Just say the word 🚀

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = count()**
- **No custom aggregation / reduce / collect logic**
- Only **built-in Stream operations**
- Java **8+ compatible**
- One clear intent per class

✅"No custom logic allowed" respected:
count() is the **only terminal**, no reducers, no collectors, no mutation.

---

1 filter() boolean checks + count()

```
import java.util.*;

public class FilterBooleanCount {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(10, 15, 20, 25);

      long count = numbers.stream()
            .filter(n -> n % 2 == 0)
            .count();

      System.out.println(count);
   }
}
```

---

2 filter() null checks + count()

```
import java.util.*;

public class FilterNullCount {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Amit");

      long count = names.stream()
            .filter(Objects::nonNull)
            .count();

      System.out.println(count);
   }
}
```

---

3 filter() comparisons + count()

```
import java.util.*;

public class FilterComparisonCount {
   public static void main(String[] args) {
      List<Integer> salaries = List.of(20000, 40000, 60000);

      long count = salaries.stream()
            .filter(s -> s >= 30000)
            .count();

      System.out.println(count);
   }
}
```

---

4 filter() method call returning boolean + count()

```
import java.util.*;

public class FilterMethodCount {

   static class User {
      boolean active;
      User(boolean active) { this.active = active; }
      boolean isActive() { return active; }
   }

   public static void main(String[] args) {
      List<User> users = List.of(
            new User(true),
            new User(false),
            new User(true)
      );

      long count = users.stream()
            .filter(User::isActive)
            .count();

      System.out.println(count);
   }
}
```

---

5 distinct() rely on equals() / hashCode() + count()

```
import java.util.*;

public class DistinctEqualsCount {
```

```
    static class Person {
        String name;
        Person(String name) { this.name = name; }

        @Override
        public boolean equals(Object o) {
            return o instanceof Person p && name.equals(p.name);
        }

        @Override
        public int hashCode() {
            return Objects.hash(name);
        }
    }

    public static void main(String[] args) {
        List<Person> people = List.of(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        );

        long count = people.stream()
                .distinct()
                .count();

        System.out.println(count);
    }
}
```

---

6 distinct() immutable objects + count()

```
import java.util.*;

public class DistinctImmutableCount {
    public static void main(String[] args) {
        List<String> values = List.of("Java", "Java", "Python");

        long count = values.stream()
                .distinct()
                .count();

        System.out.println(count);
    }
}
```

---

7 map() transform objects + count()

```
import java.util.*;

public class MapTransformCount {
   public static void main(String[] args) {
      List<String> names = List.of("anurag", "amit");

      long count = names.stream()
            .map(String::toUpperCase)
            .count();

      System.out.println(count);
   }
}
```

---

8 map() create new objects + count()

```
import java.util.*;

public class MapCreateObjectCount {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Amit");

      long count = names.stream()
            .map(UserDTO::new)
            .count();

      System.out.println(count);
   }
}
```

---

9 map() call pure functions + count()

```
import java.util.*;

public class MapPureFunctionCount {

   static int square(int n) { return n * n; }

   public static void main(String[] args) {
      List<Integer> numbers = List.of(2, 3, 4);
```

```
        long count = numbers.stream()
                .map(MapPureFunctionCount::square)
                .count();

        System.out.println(count);
    }
}
```

---

10 flatMap() return Stream + count()

```
import java.util.*;

public class FlatMapStreamCount {
    public static void main(String[] args) {
        List<List<Integer>> lists = List.of(
                List.of(1, 2),
                List.of(3, 4)
        );

        long count = lists.stream()
                .flatMap(List::stream)
                .count();

        System.out.println(count);
    }
}
```

---

11 flatMap() split data + count()

```
import java.util.*;

public class FlatMapSplitCount {
    public static void main(String[] args) {
        List<String> lines = List.of("Java Stream", "Flat Map");

        long count = lines.stream()
                .flatMap(s -> Arrays.stream(s.split(" ")))
                .count();

        System.out.println(count);
    }
}
```

---

12 flatMap() expand collections + count()

```
import java.util.*;
```

```
public class FlatMapExpandCount {
   public static void main(String[] args) {
      Map<String, List<String>> data = Map.of(
            "A", List.of("1", "2"),
            "B", List.of("3")
      );

      long count = data.values().stream()
            .flatMap(Collection::stream)
            .count();

      System.out.println(count);
   }
}
```

---

13 sorted() natural ordering + count()

```
import java.util.*;

public class SortedNaturalCount {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(5, 1, 3);

      long count = numbers.stream()
            .sorted()
            .count();

      System.out.println(count);
   }
}
```

---

14 sorted() custom comparator + count()

```
import java.util.*;

public class SortedCustomCount {
   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Ravi", "Amit");

      long count = names.stream()
            .sorted(Comparator.reverseOrder())
            .count();

      System.out.println(count);
   }
}
```

---

15 sorted() multi-field compare + count()

```
import java.util.*;

public class SortedMultiFieldCount {

   static class Employee {
      String name;
      int age;
      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {
      List<Employee> emps = List.of(
            new Employee("Anurag", 30),
            new Employee("Anurag", 25),
            new Employee("Ravi", 35)
      );

      long count = emps.stream()
            .sorted(Comparator
                  .comparing((Employee e) -> e.name)
                  .thenComparingInt(e -> e.age))
            .count();

      System.out.println(count);
   }
}
```

---

16 peek() logging + count()

```
import java.util.*;

public class PeekLoggingCount {
   public static void main(String[] args) {
      long count = List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Peek: " + n))
            .count();

      System.out.println(count);
   }
}
```

---

17 peek() debugging + count()

```
import java.util.*;

public class PeekDebugCount {
   public static void main(String[] args) {
      long count = List.of(10, 20, 30).stream()
            .peek(n -> {
               if (n == 20) {
                  System.out.println("Debug hit");
               }
            })
            .count();

      System.out.println(count);
   }
}
```

---

18 peek() metrics + count()

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsCount {
   public static void main(String[] args) {
      AtomicInteger counter = new AtomicInteger();

      long count = List.of(5, 10, 15).stream()
            .peek(n -> counter.incrementAndGet())
            .count();

      System.out.println("Stream count: " + count);
      System.out.println("Metric count: " + counter.get());
   }
}
```

Key Takeaways (important)

- count() **ignores element values** → only cardinality matters
- Intermediate ops may **transform**, but result is **just a number**
- sorted() + count() is usually **wasteful** (ordering unused)
- peek() + count() executes side-effects but returns only size

If you want next:

- findFirst() / findAny() matrix
- anyMatch / allMatch / noneMatch

- □ Cost analysis of useless intermediates
- □ Parallel count() behavior

Just tell me □

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = findFirst()**
- Purpose = **conditional check before the first matching element**
- Demonstrates **short-circuiting** (stream stops after first match)
- Java **8+ compatible**
- No collectors, no reduce, no extra logic

□ findFirst() returns Optional<T> and respects **encounter order**.

---

1 filter() boolean checks + findFirst()

```
import java.util.*;

public class FilterBooleanFindFirst {
   public static void main(String[] args) {
      Optional<Integer> firstEven =
            List.of(11, 14, 18).stream()
                  .filter(n -> n % 2 == 0)
                  .findFirst();

      System.out.println(firstEven.orElse(-1));
   }
}
```

---

2 filter() null checks + findFirst()

```
import java.util.*;

public class FilterNullFindFirst {
   public static void main(String[] args) {
      Optional<String> firstNonNull =
            Arrays.asList(null, "Anurag", "Amit").stream()
                  .filter(Objects::nonNull)
                  .findFirst();

      System.out.println(firstNonNull.orElse("NONE"));
   }
}
```

---

3 filter() comparisons + findFirst()

```
import java.util.*;

public class FilterComparisonFindFirst {
   public static void main(String[] args) {
      Optional<Integer> firstHighSalary =
            List.of(20000, 45000, 60000).stream()
                  .filter(s -> s >= 40000)
                  .findFirst();

      System.out.println(firstHighSalary.orElse(0));
   }
}
```

---

4 filter() method call returning boolean + findFirst()

```
import java.util.*;

public class FilterMethodFindFirst {

   static class User {
      String name;
      boolean active;
      User(String name, boolean active) {
         this.name = name; this.active = active;
      }
      boolean isActive() { return active; }
   }

   public static void main(String[] args) {
      Optional<User> firstActive =
            List.of(
                  new User("A", false),
                  new User("B", true),
                  new User("C", true)
            ).stream()
             .filter(User::isActive)
             .findFirst();

      System.out.println(firstActive.map(u -> u.name).orElse("NONE"));
   }
}
```

---

5 distinct() rely on equals() / hashCode() + findFirst()

```
import java.util.*;
```

```
public class DistinctEqualsFindFirst {

   static class Person {
      String name;
      Person(String name) { this.name = name; }
      @Override public boolean equals(Object o) {
         return o instanceof Person p && name.equals(p.name);
      }
      @Override public int hashCode() { return Objects.hash(name); }
   }

   public static void main(String[] args) {
      Optional<Person> firstUnique =
            List.of(
                  new Person("Anurag"),
                  new Person("Anurag"),
                  new Person("Ravi")
            ).stream()
             .distinct()
             .findFirst();

      System.out.println(firstUnique.map(p -> p.name).orElse("NONE"));
   }
}
```

---

6 distinct() immutable objects + findFirst()

```
import java.util.*;

public class DistinctImmutableFindFirst {
   public static void main(String[] args) {
      Optional<String> firstUnique =
            List.of("Java", "Java", "Go").stream()
                  .distinct()
                  .findFirst();

      System.out.println(firstUnique.orElse("NONE"));
   }
}
```

---

7 map() transform objects + findFirst()

```
import java.util.*;

public class MapTransformFindFirst {
   public static void main(String[] args) {
```

```
    Optional<String> firstUpper =
        List.of("anurag", "amit").stream()
            .map(String::toUpperCase)
            .findFirst();

    System.out.println(firstUpper.orElse("NONE"));
  }
}
```

---

8 map() create new objects + findFirst()

```
import java.util.*;

public class MapCreateObjectFindFirst {

  static class UserDTO {
    String name;
    UserDTO(String name) { this.name = name; }
  }

  public static void main(String[] args) {
    Optional<UserDTO> firstDTO =
        List.of("Anurag", "Amit").stream()
            .map(UserDTO::new)
            .findFirst();

    System.out.println(firstDTO.map(d -> d.name).orElse("NONE"));
  }
}
```

---

9 map() call pure functions + findFirst()

```
import java.util.*;

public class MapPureFunctionFindFirst {

  static int square(int n) { return n * n; }

  public static void main(String[] args) {
    Optional<Integer> firstSquare =
        List.of(2, 3, 4).stream()
            .map(MapPureFunctionFindFirst::square)
            .findFirst();

    System.out.println(firstSquare.orElse(0));
  }
}
```

---

10 flatMap() return Stream + findFirst()

```
import java.util.*;

public class FlatMapStreamFindFirst {
   public static void main(String[] args) {
      Optional<Integer> first =
            List.of(
                  List.of(10, 20),
                  List.of(30)
            ).stream()
             .flatMap(List::stream)
             .findFirst();

      System.out.println(first.orElse(0));
   }
}
```

---

11 flatMap() split data + findFirst()

```
import java.util.*;

public class FlatMapSplitFindFirst {
   public static void main(String[] args) {
      Optional<String> firstWord =
            List.of("Java Stream", "Flat Map").stream()
                  .flatMap(s -> Arrays.stream(s.split(" ")))
                  .findFirst();

      System.out.println(firstWord.orElse("NONE"));
   }
}
```

---

12 flatMap() expand collections + findFirst()

```
import java.util.*;

public class FlatMapExpandFindFirst {
   public static void main(String[] args) {
      Optional<String> firstValue =
            Map.of(
                  "A", List.of("1", "2"),
                  "B", List.of("3")
            ).values().stream()
             .flatMap(Collection::stream)
```

```
            .findFirst();

        System.out.println(firstValue.orElse("NONE"));
    }
}
```

---

13 sorted() natural ordering + findFirst()

```
import java.util.*;

public class SortedNaturalFindFirst {
    public static void main(String[] args) {
        Optional<Integer> smallest =
            List.of(5, 1, 3).stream()
                .sorted()
                .findFirst();

        System.out.println(smallest.orElse(0));
    }
}
```

---

14 sorted() custom comparator + findFirst()

```
import java.util.*;

public class SortedCustomFindFirst {
    public static void main(String[] args) {
        Optional<String> firstDesc =
            List.of("Anurag", "Ravi", "Amit").stream()
                .sorted(Comparator.reverseOrder())
                .findFirst();

        System.out.println(firstDesc.orElse("NONE"));
    }
}
```

---

15 sorted() multi-field compare + findFirst()

```
import java.util.*;

public class SortedMultiFieldFindFirst {

    static class Employee {
        String name;
        int age;
        Employee(String name, int age) {
```

```
        this.name = name; this.age = age;
    }
  }

  public static void main(String[] args) {
    Optional<Employee> first =
        List.of(
            new Employee("Anurag", 30),
            new Employee("Anurag", 25),
            new Employee("Ravi", 35)
        ).stream()
         .sorted(Comparator
             .comparing((Employee e) -> e.name)
             .thenComparingInt(e -> e.age))
         .findFirst();

    System.out.println(first.map(e -> e.name + " " + e.age).orElse("NONE"));
  }
}
```

---

16 peek() logging + findFirst()

```
import java.util.*;

public class PeekLoggingFindFirst {
  public static void main(String[] args) {
    Optional<Integer> first =
        List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Peek: " + n))
            .findFirst();

    System.out.println(first.orElse(0));
  }
}
```

---

17 peek() debugging + findFirst()

```
import java.util.*;

public class PeekDebugFindFirst {
  public static void main(String[] args) {
    Optional<Integer> first =
        List.of(10, 20, 30).stream()
            .peek(n -> {
              if (n == 20) System.out.println("Debug hit");
            })
            .findFirst();
```

```
        System.out.println(first.orElse(0));
    }
}
```

---

18 peek() metrics + findFirst()

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsFindFirst {
    public static void main(String[] args) {
        AtomicInteger counter = new AtomicInteger();

        Optional<Integer> first =
                List.of(5, 10, 15).stream()
                        .peek(n -> counter.incrementAndGet())
                        .findFirst();

        System.out.println("First: " + first.orElse(0));
        System.out.println("Processed elements: " + counter.get());
    }
}
```

 Key Takeaways

- findFirst() **short-circuits** after first match
- Respects **encounter order**
- Safer than findAny() when order matters
- peek() runs **only until first element is found**

If you want next:

- findAny() vs findFirst()
- anyMatch / allMatch / noneMatch
- Parallel stream behavior differences
- Performance pitfalls

Just say

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = allMatch()**
- Purpose = **validation logic** (returns true only if **all elements** satisfy the predicate)
- Demonstrates **short-circuiting** (stops on first failure)
- Java **8+** compatible
- No collectors / reduce / custom aggregation

---

1 filter() boolean checks + allMatch() (Validation)

```
import java.util.*;

public class FilterBooleanAllMatch {
   public static void main(String[] args) {
      boolean valid = List.of(2, 4, 6, 8).stream()
            .filter(n -> n > 0)
            .allMatch(n -> n % 2 == 0);

      System.out.println(valid);
   }
}
```

---

2 filter() null checks + allMatch() (Validation)

```
import java.util.*;

public class FilterNullAllMatch {
   public static void main(String[] args) {
      boolean valid = Arrays.asList("Anurag", "Amit").stream()
            .filter(Objects::nonNull)
            .allMatch(s -> !s.isEmpty());

      System.out.println(valid);
   }
}
```

---

3 filter() comparisons + allMatch() (Validation)

```
import java.util.*;

public class FilterComparisonAllMatch {
   public static void main(String[] args) {
      boolean valid = List.of(30, 40, 50).stream()
            .filter(age -> age >= 18)
```

```
            .allMatch(age -> age <= 60);

        System.out.println(valid);
    }
}
```

---

4 filter() method calls returning boolean + allMatch() (Validation)

```
import java.util.*;

public class FilterMethodAllMatch {

    static class User {
        boolean active;
        User(boolean active) { this.active = active; }
        boolean isActive() { return active; }
    }

    public static void main(String[] args) {
        boolean valid = List.of(
                new User(true),
                new User(true)
        ).stream()
         .filter(User::isActive)
         .allMatch(User::isActive);

        System.out.println(valid);
    }
}
```

---

5 distinct() rely on equals() / hashCode() + allMatch() (Validation)

```
import java.util.*;

public class DistinctEqualsAllMatch {

    static class Person {
        String name;
        Person(String name) { this.name = name; }

        @Override public boolean equals(Object o) {
            return o instanceof Person p && name.equals(p.name);
        }
        @Override public int hashCode() {
            return Objects.hash(name);
        }
    }
```

```
    public static void main(String[] args) {
        boolean valid = List.of(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        ).stream()
         .distinct()
         .allMatch(p -> p.name.length() >= 4);

        System.out.println(valid);
    }
}
```

---

6 distinct() immutable objects + allMatch() (Validation)

```
import java.util.*;

public class DistinctImmutableAllMatch {
    public static void main(String[] args) {
        boolean valid = List.of("Java", "Java", "Go").stream()
                .distinct()
                .allMatch(s -> s.length() >= 2);

        System.out.println(valid);
    }
}
```

---

7 map() transform objects + allMatch() (Validation)

```
import java.util.*;

public class MapTransformAllMatch {
    public static void main(String[] args) {
        boolean valid = List.of("anurag", "amit").stream()
                .map(String::toUpperCase)
                .allMatch(s -> s.equals(s.toUpperCase()));

        System.out.println(valid);
    }
}
```

---

8 map() create new objects + allMatch() (Validation)

```
import java.util.*;
```

```
public class MapCreateObjectAllMatch {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {
      boolean valid = List.of("Anurag", "Amit").stream()
            .map(UserDTO::new)
            .allMatch(dto -> dto.name.length() >= 4);

      System.out.println(valid);
   }
}
```

---

9 map() call pure functions + allMatch() (Validation)

```
import java.util.*;

public class MapPureFunctionAllMatch {

   static int square(int n) { return n * n; }

   public static void main(String[] args) {
      boolean valid = List.of(2, 3, 4).stream()
            .map(MapPureFunctionAllMatch::square)
            .allMatch(n -> n > 0);

      System.out.println(valid);
   }
}
```

---

10 flatMap() return Stream + allMatch() (Validation)

```
import java.util.*;

public class FlatMapStreamAllMatch {
   public static void main(String[] args) {
      boolean valid = List.of(
            List.of(1, 2),
            List.of(3, 4)
      ).stream()
       .flatMap(List::stream)
       .allMatch(n -> n > 0);

      System.out.println(valid);
```

```
    }
}
```

---

11 flatMap() split data + allMatch() (Validation)

```
import java.util.*;

public class FlatMapSplitAllMatch {
    public static void main(String[] args) {
        boolean valid = List.of("Java Stream", "Flat Map").stream()
                .flatMap(s -> Arrays.stream(s.split(" ")))
                .allMatch(word -> word.length() >= 3);

        System.out.println(valid);
    }
}
```

---

12 flatMap() expand collections + allMatch() (Validation)

```
import java.util.*;

public class FlatMapExpandAllMatch {
    public static void main(String[] args) {
        boolean valid = Map.of(
                "A", List.of("1", "2"),
                "B", List.of("3")
        ).values().stream()
         .flatMap(Collection::stream)
         .allMatch(v -> !v.isEmpty());

        System.out.println(valid);
    }
}
```

---

13 sorted() natural ordering + allMatch() (Validation)

```
import java.util.*;

public class SortedNaturalAllMatch {
    public static void main(String[] args) {
        boolean valid = List.of(1, 2, 3).stream()
                .sorted()
                .allMatch(n -> n >= 1);

        System.out.println(valid);
    }
```

```
}
```

---

14 sorted() custom comparator + allMatch() (Validation)

```
import java.util.*;

public class SortedCustomAllMatch {
   public static void main(String[] args) {
      boolean valid = List.of("Anurag", "Ravi", "Amit").stream()
            .sorted(Comparator.reverseOrder())
            .allMatch(name -> name.length() >= 4);

      System.out.println(valid);
   }
}
```

---

15 sorted() multi-field compare + allMatch() (Validation)

```
import java.util.*;

public class SortedMultiFieldAllMatch {

   static class Employee {
      String name;
      int age;
      Employee(String name, int age) {
         this.name = name; this.age = age;
      }
   }

   public static void main(String[] args) {
      boolean valid = List.of(
            new Employee("Anurag", 30),
            new Employee("Anurag", 25),
            new Employee("Ravi", 35)
      ).stream()
       .sorted(Comparator
             .comparing((Employee e) -> e.name)
             .thenComparingInt(e -> e.age))
       .allMatch(e -> e.age >= 18);

      System.out.println(valid);
   }
}
```

---

16 peek() logging + allMatch() (Validation)

```
import java.util.*;

public class PeekLoggingAllMatch {
   public static void main(String[] args) {
      boolean valid = List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Checking: " + n))
            .allMatch(n -> n > 0);

      System.out.println(valid);
   }
}
```

---

17 peek() debugging + allMatch() (Validation)

```
import java.util.*;

public class PeekDebugAllMatch {
   public static void main(String[] args) {
      boolean valid = List.of(10, 20, 30).stream()
            .peek(n -> {
               if (n == 20) System.out.println("Debug hit");
            })
            .allMatch(n -> n >= 10);

      System.out.println(valid);
   }
}
```

---

18 peek() metrics + allMatch() (Validation)

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsAllMatch {
   public static void main(String[] args) {
      AtomicInteger counter = new AtomicInteger();

      boolean valid = List.of(5, 10, 15).stream()
            .peek(n -> counter.incrementAndGet())
            .allMatch(n -> n > 0);

      System.out.println("Valid: " + valid);
      System.out.println("Processed elements: " + counter.get());
   }
}
```

 Key Takeaways

- allMatch() is **short-circuiting** → stops on first failure
- Returns true for **empty streams** (important interview point)
- Use for **validation rules**, not aggregation
- peek() executes only until a failure occurs

If you want next:

- anyMatch() vs allMatch() vs noneMatch()
- Parallel stream behavior
- Validation anti-patterns

Just say

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = allMatch()**
- Purpose = **rule checks** (business / validation rules)
- **Short-circuiting behavior** is intentional
- Java **8+**
- No collectors, no reduce, no side-effect mutation (except peek demos)

---

1 filter() boolean checks + allMatch() — Rule checks

```
import java.util.*;

public class FilterBooleanAllMatchRule {
   public static void main(String[] args) {
      boolean rulePassed = List.of(2, 4, 6, 8).stream()
            .filter(n -> n > 0)
            .allMatch(n -> n % 2 == 0);

      System.out.println(rulePassed);
   }
}
```

---

2 filter() null checks + allMatch() — Rule checks

```
import java.util.*;

public class FilterNullAllMatchRule {
   public static void main(String[] args) {
      boolean rulePassed = Arrays.asList("Anurag", "Amit").stream()
            .filter(Objects::nonNull)
            .allMatch(name -> name.length() >= 4);

      System.out.println(rulePassed);
   }
}
```

---

3 filter() comparisons + allMatch() — Rule checks

```
import java.util.*;

public class FilterComparisonAllMatchRule {
   public static void main(String[] args) {
      boolean rulePassed = List.of(25, 30, 40).stream()
            .filter(age -> age >= 18)
```

```
            .allMatch(age -> age <= 60);

        System.out.println(rulePassed);
    }
}
```

---

4 filter() method calls returning boolean + allMatch() — Rule checks

```
import java.util.*;

public class FilterMethodAllMatchRule {

    static class User {
        boolean verified;
        User(boolean verified) { this.verified = verified; }
        boolean isVerified() { return verified; }
    }

    public static void main(String[] args) {
        boolean rulePassed = List.of(
                new User(true),
                new User(true)
        ).stream()
         .filter(User::isVerified)
         .allMatch(User::isVerified);

        System.out.println(rulePassed);
    }
}
```

---

5 distinct() rely on equals() / hashCode() + allMatch() — Rule checks

```
import java.util.*;

public class DistinctEqualsAllMatchRule {

    static class Person {
        String email;
        Person(String email) { this.email = email; }

        @Override
        public boolean equals(Object o) {
            return o instanceof Person p && email.equals(p.email);
        }

        @Override
        public int hashCode() {
```

```
        return Objects.hash(email);
    }
  }

  public static void main(String[] args) {
    boolean rulePassed = List.of(
          new Person("a@mail.com"),
          new Person("a@mail.com"),
          new Person("b@mail.com")
    ).stream()
     .distinct()
     .allMatch(p -> p.email.contains("@"));

    System.out.println(rulePassed);
  }
}
```

---

6 distinct() immutable objects + allMatch() — Rule checks

```
import java.util.*;

public class DistinctImmutableAllMatchRule {
  public static void main(String[] args) {
    boolean rulePassed = List.of("Java", "Java", "Python").stream()
          .distinct()
          .allMatch(lang -> lang.length() >= 4);

    System.out.println(rulePassed);
  }
}
```

---

7 map() transform objects + allMatch() — Rule checks

```
import java.util.*;

public class MapTransformAllMatchRule {
  public static void main(String[] args) {
    boolean rulePassed = List.of("anurag", "amit").stream()
          .map(String::toUpperCase)
          .allMatch(s -> s.equals(s.toUpperCase()));

    System.out.println(rulePassed);
  }
}
```

---

8 map() create new objects + allMatch() — Rule checks

```
import java.util.*;

public class MapCreateObjectAllMatchRule {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {
      boolean rulePassed = List.of("Anurag", "Amit").stream()
            .map(UserDTO::new)
            .allMatch(dto -> dto.name.length() >= 4);

      System.out.println(rulePassed);
   }
}
```

---

9 map() call pure functions + allMatch() — Rule checks

```
import java.util.*;

public class MapPureFunctionAllMatchRule {

   static int normalize(int n) { return n * 10; }

   public static void main(String[] args) {
      boolean rulePassed = List.of(1, 2, 3).stream()
            .map(MapPureFunctionAllMatchRule::normalize)
            .allMatch(n -> n >= 10);

      System.out.println(rulePassed);
   }
}
```

---

10 flatMap() return Stream + allMatch() — Rule checks

```
import java.util.*;

public class FlatMapStreamAllMatchRule {
   public static void main(String[] args) {
      boolean rulePassed = List.of(
            List.of(1, 2),
            List.of(3, 4)
      ).stream()
       .flatMap(List::stream)
       .allMatch(n -> n > 0);
```

```
        System.out.println(rulePassed);
    }
}
```

---

11 flatMap() split data + allMatch() — Rule checks

```
import java.util.*;

public class FlatMapSplitAllMatchRule {
    public static void main(String[] args) {
        boolean rulePassed = List.of("Java Stream", "Flat Map").stream()
                .flatMap(s -> Arrays.stream(s.split(" ")))
                .allMatch(word -> word.length() >= 3);

        System.out.println(rulePassed);
    }
}
```

---

12 flatMap() expand collections + allMatch() — Rule checks

```
import java.util.*;

public class FlatMapExpandAllMatchRule {
    public static void main(String[] args) {
        boolean rulePassed = Map.of(
                "A", List.of("1", "2"),
                "B", List.of("3")
        ).values().stream()
         .flatMap(Collection::stream)
         .allMatch(v -> !v.isEmpty());

        System.out.println(rulePassed);
    }
}
```

---

13 sorted() natural ordering + allMatch() — Rule checks

```
import java.util.*;

public class SortedNaturalAllMatchRule {
    public static void main(String[] args) {
        boolean rulePassed = List.of(1, 2, 3).stream()
                .sorted()
                .allMatch(n -> n >= 1);
```

```
        System.out.println(rulePassed);
    }
}
```

---

14 sorted() custom comparator + allMatch() — Rule checks

```
import java.util.*;

public class SortedCustomAllMatchRule {
    public static void main(String[] args) {
        boolean rulePassed = List.of("Anurag", "Ravi", "Amit").stream()
                .sorted(Comparator.reverseOrder())
                .allMatch(name -> name.length() >= 4);

        System.out.println(rulePassed);
    }
}
```

---

15 sorted() multi-field compare + allMatch() — Rule checks

```
import java.util.*;

public class SortedMultiFieldAllMatchRule {

    static class Employee {
        String name;
        int age;
        Employee(String name, int age) {
            this.name = name; this.age = age;
        }
    }

    public static void main(String[] args) {
        boolean rulePassed = List.of(
                new Employee("Anurag", 30),
                new Employee("Anurag", 25),
                new Employee("Ravi", 35)
        ).stream()
         .sorted(Comparator
                 .comparing((Employee e) -> e.name)
                 .thenComparingInt(e -> e.age))
         .allMatch(e -> e.age >= 18);

        System.out.println(rulePassed);
    }
}
```

---

16 peek() logging + allMatch() — Rule checks

```
import java.util.*;

public class PeekLoggingAllMatchRule {
   public static void main(String[] args) {
      boolean rulePassed = List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Checking: " + n))
            .allMatch(n -> n > 0);

      System.out.println(rulePassed);
   }
}
```

---

17 peek() debugging + allMatch() — Rule checks

```
import java.util.*;

public class PeekDebugAllMatchRule {
   public static void main(String[] args) {
      boolean rulePassed = List.of(10, 20, 30).stream()
            .peek(n -> {
               if (n == 20) System.out.println("Debug breakpoint");
            })
            .allMatch(n -> n >= 10);

      System.out.println(rulePassed);
   }
}
```

---

18 peek() metrics + allMatch() — Rule checks

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsAllMatchRule {
   public static void main(String[] args) {
      AtomicInteger processed = new AtomicInteger();

      boolean rulePassed = List.of(5, 10, 15).stream()
            .peek(n -> processed.incrementAndGet())
            .allMatch(n -> n > 0);

      System.out.println("Rule passed: " + rulePassed);
      System.out.println("Processed elements: " + processed.get());
   }
}
```

⚠ Important Rule-Check Insights

- allMatch() **fails fast** (first violation stops the stream)
- **Empty stream → true** (critical interview rule)
- Best suited for **policy / rule / constraint enforcement**
- Avoid peek() in production validation logic

If you want next:

- ⬜ anyMatch() vs allMatch() vs noneMatch() rule matrix
- ⬜ Parallel stream rule-check pitfalls
- ⬜ Real-world validation patterns (ACLs, policies)

Just say ⬜

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = anyMatch()**
- Purpose = **existence checks** (returns true if **any element** satisfies the predicate)
- Demonstrates **short-circuiting** (stops on first match)
- Java **8+** compatible
- No collectors / reduce / custom aggregation

---

1 filter() boolean checks + anyMatch() — Existence check

```
import java.util.*;

public class FilterBooleanAnyMatch {
   public static void main(String[] args) {
      boolean exists = List.of(1, 3, 4, 7).stream()
            .filter(n -> n > 0)
            .anyMatch(n -> n % 2 == 0);

      System.out.println(exists);
   }
}
```

---

2 filter() null checks + anyMatch() — Existence check

```
import java.util.*;

public class FilterNullAnyMatch {
   public static void main(String[] args) {
      boolean exists = Arrays.asList(null, "Anurag", "Amit").stream()
            .filter(Objects::nonNull)
            .anyMatch(s -> s.startsWith("A"));

      System.out.println(exists);
   }
}
```

---

3 filter() comparisons + anyMatch() — Existence check

```
import java.util.*;

public class FilterComparisonAnyMatch {
   public static void main(String[] args) {
      boolean exists = List.of(20000, 45000, 60000).stream()
            .filter(s -> s >= 30000)
```

```
            .anyMatch(s -> s > 50000);

        System.out.println(exists);
    }
}
```

---

4 filter() method calls returning boolean + anyMatch() — Existence check

```
import java.util.*;

public class FilterMethodAnyMatch {

    static class User {
        boolean active;
        User(boolean active) { this.active = active; }
        boolean isActive() { return active; }
    }

    public static void main(String[] args) {
        boolean exists = List.of(
                new User(false),
                new User(true)
        ).stream()
         .filter(User::isActive)
         .anyMatch(User::isActive);

        System.out.println(exists);
    }
}
```

---

5 distinct() rely on equals() / hashCode() + anyMatch() — Existence check

```
import java.util.*;

public class DistinctEqualsAnyMatch {

    static class Person {
        String name;
        Person(String name) { this.name = name; }

        @Override public boolean equals(Object o) {
            return o instanceof Person p && name.equals(p.name);
        }
        @Override public int hashCode() {
            return Objects.hash(name);
        }
    }
```

```
    public static void main(String[] args) {
        boolean exists = List.of(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        ).stream()
         .distinct()
         .anyMatch(p -> p.name.equals("Ravi"));

        System.out.println(exists);
    }
}
```

---

6 distinct() immutable objects + anyMatch() — Existence check

```
import java.util.*;

public class DistinctImmutableAnyMatch {
    public static void main(String[] args) {
        boolean exists = List.of("Java", "Java", "Python").stream()
                .distinct()
                .anyMatch(lang -> lang.startsWith("P"));

        System.out.println(exists);
    }
}
```

---

7 map() transform objects + anyMatch() — Existence check

```
import java.util.*;

public class MapTransformAnyMatch {
    public static void main(String[] args) {
        boolean exists = List.of("anurag", "amit").stream()
                .map(String::toUpperCase)
                .anyMatch(s -> s.equals("AMIT"));

        System.out.println(exists);
    }
}
```

---

8 map() create new objects + anyMatch() — Existence check

```
import java.util.*;
```

```
public class MapCreateObjectAnyMatch {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {
      boolean exists = List.of("Anurag", "Amit").stream()
            .map(UserDTO::new)
            .anyMatch(dto -> dto.name.equals("Amit"));

      System.out.println(exists);
   }
}
```

---

9 map() call pure functions + anyMatch() — Existence check

```
import java.util.*;

public class MapPureFunctionAnyMatch {

   static int square(int n) { return n * n; }

   public static void main(String[] args) {
      boolean exists = List.of(2, 3, 4).stream()
            .map(MapPureFunctionAnyMatch::square)
            .anyMatch(n -> n == 16);

      System.out.println(exists);
   }
}
```

---

10 flatMap() return Stream + anyMatch() — Existence check

```
import java.util.*;

public class FlatMapStreamAnyMatch {
   public static void main(String[] args) {
      boolean exists = List.of(
            List.of(1, 2),
            List.of(3, 4)
      ).stream()
       .flatMap(List::stream)
       .anyMatch(n -> n == 3);

      System.out.println(exists);
```

```
    }
}
```

---

11 flatMap() split data + anyMatch() — Existence check

```
import java.util.*;

public class FlatMapSplitAnyMatch {
    public static void main(String[] args) {
        boolean exists = List.of("Java Stream", "Flat Map").stream()
                .flatMap(s -> Arrays.stream(s.split(" ")))
                .anyMatch(word -> word.equals("Stream"));

        System.out.println(exists);
    }
}
```

---

12 flatMap() expand collections + anyMatch() — Existence check

```
import java.util.*;

public class FlatMapExpandAnyMatch {
    public static void main(String[] args) {
        boolean exists = Map.of(
                "A", List.of("1", "2"),
                "B", List.of("3")
        ).values().stream()
         .flatMap(Collection::stream)
         .anyMatch(v -> v.equals("2"));

        System.out.println(exists);
    }
}
```

---

13 sorted() natural ordering + anyMatch() — Existence check

```
import java.util.*;

public class SortedNaturalAnyMatch {
    public static void main(String[] args) {
        boolean exists = List.of(5, 1, 3).stream()
                .sorted()
                .anyMatch(n -> n == 1);

        System.out.println(exists);
    }
```

```
}
```

---

14 sorted() custom comparator + anyMatch() — Existence check

```
import java.util.*;

public class SortedCustomAnyMatch {
   public static void main(String[] args) {
      boolean exists = List.of("Anurag", "Ravi", "Amit").stream()
            .sorted(Comparator.reverseOrder())
            .anyMatch(name -> name.equals("Amit"));

      System.out.println(exists);
   }
}
```

---

15 sorted() multi-field compare + anyMatch() — Existence check

```
import java.util.*;

public class SortedMultiFieldAnyMatch {

   static class Employee {
      String name;
      int age;
      Employee(String name, int age) {
         this.name = name; this.age = age;
      }
   }

   public static void main(String[] args) {
      boolean exists = List.of(
            new Employee("Anurag", 30),
            new Employee("Anurag", 25),
            new Employee("Ravi", 35)
      ).stream()
       .sorted(Comparator
             .comparing((Employee e) -> e.name)
             .thenComparingInt(e -> e.age))
       .anyMatch(e -> e.age == 25);

      System.out.println(exists);
   }
}
```

---

16 peek() logging + anyMatch() — Existence check

```
import java.util.*;

public class PeekLoggingAnyMatch {
   public static void main(String[] args) {
      boolean exists = List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Checking: " + n))
            .anyMatch(n -> n == 2);

      System.out.println(exists);
   }
}
```

---

17 peek() debugging + anyMatch() — Existence check

```
import java.util.*;

public class PeekDebugAnyMatch {
   public static void main(String[] args) {
      boolean exists = List.of(10, 20, 30).stream()
            .peek(n -> {
               if (n == 20) System.out.println("Debug breakpoint");
            })
            .anyMatch(n -> n == 20);

      System.out.println(exists);
   }
}
```

---

18 peek() metrics + anyMatch() — Existence check

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsAnyMatch {
   public static void main(String[] args) {
      AtomicInteger processed = new AtomicInteger();

      boolean exists = List.of(5, 10, 15).stream()
            .peek(n -> processed.incrementAndGet())
            .anyMatch(n -> n == 10);

      System.out.println("Exists: " + exists);
      System.out.println("Processed elements: " + processed.get());
   }
}
```

Key Takeaways

- anyMatch() **short-circuits** on the first match
- Returns **false** for empty streams
- Best suited for **existence / presence checks**
- peek() executes **only until a match is found**

If you want next:

- noneMatch() existence-absence patterns
- findAny() vs anyMatch()
- Parallel stream behavior & pitfalls

Just say

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = noneMatch()**
- Purpose = **negative validation** (assert *no element* violates a rule)
- **Short-circuiting**: stops on the first violation
- Java **8+**, no collectors/reduce

---

1 filter() boolean checks + noneMatch() — Negative validation

```
import java.util.*;

public class FilterBooleanNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of(2, 4, 6, 8).stream()
            .filter(n -> n > 0)
            .noneMatch(n -> n % 2 != 0);                    // no odd numbers

      System.out.println(ok);
   }
}
```

---

2 filter() null checks + noneMatch() — Negative validation

```
import java.util.*;

public class FilterNullNoneMatch {
   public static void main(String[] args) {
      boolean ok = Arrays.asList("Anurag", "Amit").stream()
            .filter(Objects::nonNull)
            .noneMatch(String::isEmpty);                    // no empty strings

      System.out.println(ok);
   }
}
```

---

3 filter() comparisons + noneMatch() — Negative validation

```
import java.util.*;

public class FilterComparisonNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of(25, 30, 40).stream()
            .filter(age -> age >= 18)
            .noneMatch(age -> age > 60);                    // no seniors
```

```
    System.out.println(ok);
  }
}
```

---

4 filter() method calls returning boolean + noneMatch() — Negative validation

```
import java.util.*;

public class FilterMethodNoneMatch {

  static class User {
    boolean banned;
    User(boolean banned) { this.banned = banned; }
    boolean isBanned() { return banned; }
  }

  public static void main(String[] args) {
    boolean ok = List.of(new User(false), new User(false)).stream()
        .filter(u -> true)
        .noneMatch(User::isBanned);                    // no banned users

    System.out.println(ok);
  }
}
```

---

5 distinct() rely on equals() / hashCode() + noneMatch() — Negative validation

```
import java.util.*;

public class DistinctEqualsNoneMatch {

  static class Person {
    String email;
    Person(String email) { this.email = email; }
    @Override public boolean equals(Object o) {
      return o instanceof Person p && email.equals(p.email);
    }
    @Override public int hashCode() { return Objects.hash(email); }
  }

  public static void main(String[] args) {
    boolean ok = List.of(
        new Person("a@mail.com"),
        new Person("a@mail.com"),
        new Person("b@mail.com")
    ).stream()
```

```
        .distinct()
        .noneMatch(p -> p.email.endsWith("@spam.com"));

        System.out.println(ok);
    }
}
```

---

6 distinct() immutable objects + noneMatch() — Negative validation

```
import java.util.*;

public class DistinctImmutableNoneMatch {
    public static void main(String[] args) {
        boolean ok = List.of("Java", "Java", "Python").stream()
                .distinct()
                .noneMatch(lang -> lang.length() < 3);

        System.out.println(ok);
    }
}
```

---

7 map() transform objects + noneMatch() — Negative validation

```
import java.util.*;

public class MapTransformNoneMatch {
    public static void main(String[] args) {
        boolean ok = List.of("anurag", "amit").stream()
                .map(String::toUpperCase)
                .noneMatch(s -> s.contains(" "));                    // no spaces

        System.out.println(ok);
    }
}
```

---

8 map() create new objects + noneMatch() — Negative validation

```
import java.util.*;

public class MapCreateObjectNoneMatch {

    static class UserDTO {
        String name;
        UserDTO(String name) { this.name = name; }
    }
```

```
    public static void main(String[] args) {
        boolean ok = List.of("Anurag", "Amit").stream()
                .map(UserDTO::new)
                .noneMatch(dto -> dto.name.isBlank());

        System.out.println(ok);
    }
}
```

---

9 map() call pure functions + noneMatch() — Negative validation

```
import java.util.*;

public class MapPureFunctionNoneMatch {

    static int normalize(int n) { return n * 10; }

    public static void main(String[] args) {
        boolean ok = List.of(1, 2, 3).stream()
                .map(MapPureFunctionNoneMatch::normalize)
                .noneMatch(n -> n <= 0);

        System.out.println(ok);
    }
}
```

---

10 flatMap() return Stream + noneMatch() — Negative validation

```
import java.util.*;

public class FlatMapStreamNoneMatch {
    public static void main(String[] args) {
        boolean ok = List.of(
                List.of(1, 2),
                List.of(3, 4)
        ).stream()
         .flatMap(List::stream)
         .noneMatch(n -> n < 0);

        System.out.println(ok);
    }
}
```

---

11 flatMap() split data + noneMatch() — Negative validation

```
import java.util.*;
```

```
public class FlatMapSplitNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of("Java Stream", "Flat Map").stream()
            .flatMap(s -> Arrays.stream(s.split(" ")))
            .noneMatch(word -> word.length() < 3);

      System.out.println(ok);
   }
}
```

---

12 flatMap() expand collections + noneMatch() — Negative validation

```
import java.util.*;

public class FlatMapExpandNoneMatch {
   public static void main(String[] args) {
      boolean ok = Map.of(
            "A", List.of("1", "2"),
            "B", List.of("3")
      ).values().stream()
       .flatMap(Collection::stream)
       .noneMatch(String::isEmpty);

      System.out.println(ok);
   }
}
```

---

13 sorted() natural ordering + noneMatch() — Negative validation

```
import java.util.*;

public class SortedNaturalNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of(1, 2, 3).stream()
            .sorted()
            .noneMatch(n -> n <= 0);

      System.out.println(ok);
   }
}
```

---

14 sorted() custom comparator + noneMatch() — Negative validation

```
import java.util.*;
```

```
public class SortedCustomNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of("Anurag", "Ravi", "Amit").stream()
            .sorted(Comparator.reverseOrder())
            .noneMatch(name -> name.length() < 4);

      System.out.println(ok);
   }
}
```

---

15 sorted() multi-field compare + noneMatch() — Negative validation

```
import java.util.*;

public class SortedMultiFieldNoneMatch {

   static class Employee {
      String name; int age;
      Employee(String name, int age) { this.name = name; this.age = age; }
   }

   public static void main(String[] args) {
      boolean ok = List.of(
            new Employee("Anurag", 30),
            new Employee("Anurag", 25),
            new Employee("Ravi", 35)
      ).stream()
       .sorted(Comparator.comparing((Employee e) -> e.name).thenComparingInt(e -> e.age))
       .noneMatch(e -> e.age < 18);

      System.out.println(ok);
   }
}
```

---

16 peek() logging + noneMatch() — Negative validation

```
import java.util.*;

public class PeekLoggingNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Checking: " + n))
            .noneMatch(n -> n < 0);

      System.out.println(ok);
   }
}
```

---

17 peek() debugging + noneMatch() — Negative validation

```
import java.util.*;

public class PeekDebugNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of(10, 20, 30).stream()
            .peek(n -> { if (n == 20) System.out.println("Debug hit"); })
            .noneMatch(n -> n < 10);

      System.out.println(ok);
   }
}
```

---

18 peek() metrics + noneMatch() — Negative validation

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsNoneMatch {
   public static void main(String[] args) {
      AtomicInteger seen = new AtomicInteger();

      boolean ok = List.of(5, 10, 15).stream()
            .peek(n -> seen.incrementAndGet())
            .noneMatch(n -> n <= 0);

      System.out.println("OK: " + ok);
      System.out.println("Processed: " + seen.get());
   }
}
```

Key Takeaways

- noneMatch() is ideal for **"no violations exist"** checks.
- **Short-circuits** on the first violation.
- Returns **true for empty streams** (important rule).
- Avoid sorted()/peek() unless necessary—ordering/side-effects don't change the boolean result.

Want the **noneMatch() vs allMatch() vs anyMatch() decision table** or **parallel stream caveats** next?